Digital Contracting Service

Technical Manual

As of 2026-08-05

DCS Technical Manual

This manual explains the Digital Contracting Service (DCS) at system level: structure, flows, interfaces and operational behaviour. It is written for developers, operators and integrators who want to understand, operate or integrate the system. The code is the source of truth.

Table of contents

Scope relative to the other documents

This manual explains. It deliberately contains no commands, no value tables and no step-by-step procedures.

Question	Document
How is the system built, how does a process flow through it, what does a signal mean?	This manual
How do I install, configure, upgrade and uninstall an instance?	`docs/deployment-guide.md`
How do I back up and restore data?	`docs/backup-integration-guide.md`
How do I use the system as a business user?	`docs/user-manual/`
Why was something decided this way?	The ADRs under `docs/`, see Appendix B

Concrete values, commands and configuration references live in exactly one place, the deployment guide. Where this manual would need such a value, it links there.

Reading recommendations

- **New to the project:** Chapters 01 and 02, then Chapter 04 with the system's central flow.
- **Operators:** Chapter 02, then Chapter 10; Chapter 09 for the audit trail and the incident view. Executable procedures are in the deployment guide.
- **Integrators (target systems, peer instances):** Chapters 02, 07, 08 and Appendix A.
- **Security/compliance review:** Chapters 05, 06, 09 and Appendix B.

01 System Overview

What the DCS is

The **Digital Contracting Service (DCS)** is a federated, cross-federation-capable system for secure B2B communication, negotiation and automated processing of digital contracts; it can be hosted on premise. The central idea: **a digital contract is more than a signed document. It is an interoperable, machine-interpretable and cryptographically provable agreement between organizations.**

Each organization operates its own DCS instance. Instances run by different operators exchange templates and contracts without trusting each other implicitly. Trust is established and verified cryptographically per interaction.

Positioning: XFSC and FACIS

The DCS is developed within **FACIS** (Federation Architecture for Composed Infrastructure Services) in the Eclipse XFSC ecosystem:

- **Federated Catalogue:** approved templates are registered as self-descriptions and become discoverable across the federation.
- **ORCE (XFSC Orchestration Engine):** Node-RED-based orchestration layer for timestamps, archive notary, target-system integration, trust PDP and the externalized check and approval steps of the compliance subsystem.
- **EUDI Wallet / eIDAS 2.0:** end users authenticate with Verifiable Credentials (OpenID4VP); signatures follow the eIDAS formats PAdES/JAdES.

The binding requirements baseline is the SRS of the FACIS DCS; ADR-5 describes the positioning relative to the XFSC components.

Core capabilities

Semantic templates, machine-readable terms. Organizations define machine-readable contract templates on the basis of freely chosen ontologies. An instance-local Semantic Hub versions the JSON-LD contexts, SHACL shapes and validation profiles against which every document is created and checked, and provides the clause catalog. Contract terms are formally described with ODRL 2.0 and are interpretable by humans and machines alike.

Negotiation and formalized conclusion. Contracts created from templates run through a defined state machine from draft through offer, negotiation, review and approval to signing. Individual terms are negotiated as change requests. Each instance runs its own workflow with its own roles; what crosses the instance boundary is the contract document.

eIDAS-conformant signing and handover to target systems. A concluded contract is an electronically signed and machine-readable document whose signatures follow the eIDAS formats: the PDF/A-3 carries the canonical JSON-LD as an embedded attachment and is signed with PAdES/JAdES. Target systems

implement or evaluate the agreed terms automatically and report states, evidence and KPI values back to the participating instances. The contract remains a machine-interpretable agreement over its entire lifecycle.

Zero-trust identity model. Trust derives from cryptographically verifiable identities, credentials, signatures, trust lists and independently reproducible verification processes rather than from network position or organizational affiliation. End users log in with a Verifiable Presentation (OpenID4VP, SD-JWT VC, for example from an EUDI Wallet); machine callers receive their own OAuth2 clients whose rights live in a registry of the instance; each instance owns a `did:web` identity whose private keys reside exclusively in a PKCS#11 token (HSM). Issuer trust is purpose-scoped and organization-scoped (ADR-31), and every federation interaction consults a local policy endpoint, fail-closed.

Cryptographically traceable auditability. Contract, negotiation, signature and state events are recorded in a tamper-evident audit trail: hash-chained entries per resource, batched into Merkle checkpoints whose root receives an RFC 3161 timestamp and is anchored in IPFS. This makes it independently provable that a given contract or audit representation existed in a given form at a given time and has not been altered unnoticed afterwards.

Independent validation: machine-readable against human-readable. The document service pdf-core renders deterministically: the same JSON-LD payload produces byte-identical page content. Every participating instance extracts the embedded JSON-LD from an incoming signed document, independently re-renders it and compares the result with the received document. No instance depends on the word of the issuing or transmitting instance for this. A C2PA provenance chain in the document additionally makes the origin of the visible content provable.

Confidentiality and erasability. Every artifact stored in IPFS is encrypted. The key is drawn at random per contract and, at rest, can only be opened by the HSM of the respective instance. Erasing a contract in the sense of Art. 17 GDPR is the recorded destruction of these key copies on both participating instances; the Merkle proofs of the audit trail remain valid because they never depended on the readable content (ADR-28, Chapters 07 and 09).

The DCS thus combines federated identities, Verifiable Credentials, semantic contract models, ODRL terms, interoperable negotiation, eIDAS signatures, automated execution and cryptographically anchored auditability in a shared trust and communication infrastructure. Chapter 02 shows how these capabilities map to components.

02 Architecture

This chapter describes the component map of a DCS instance: which building blocks exist, what they are responsible for and who talks to whom. Chapters 03 to 09 cover the functional flows, Chapter 10 the operational behaviour, and the deployment guide the installation.

Basic structure

A DCS instance consists of three services (**backend**, **pdf-core**, **frontend**) and a set of infrastructure and XFSC components in the same deployment. The backend is a **modular monolith**: all business domains run in one process, share one PostgreSQL instance and are decoupled via an internal event bus (NATS, CloudEvents). Separate processes exist where a different technical responsibility begins: byte-level PDF work in pdf-core, orchestration and externalized check steps in ORCE, AdES signature validation in DSS.

Two cuts shape the architecture:

1. **All private keys of the instance reside in the PKCS#11 token, and only the backend accesses it** (ADR-1). Whoever needs a signature obtains it from the backend.
2. **Every persistently stored artifact is encrypted before it leaves the process** (ADR-28). The key is bound to an erasure scope: per contract, per template, or instance-wide.

Component map

DCS services

Component	Technology	Responsibility
Backend	Go, Goa v3 (design-first)	All business logic: template and contract lifecycle, negotiation, signature management, federation, audit trail, authentication, Semantic Hub, key inventory. Serves the HTTP API
pdf-core	Go	Deterministic PDF/A-3a compiler: JSON-LD to byte-stable PDF with embedded canonical JSON-LD and C2PA provenance chain; re-rendering verification, incremental amendments, provenance re-anchor. Holds **no** key material: it returns the C2PA structures to be signed to the backend. This split also enables strong horizontal scaling: PDF operations are long-running, and pdf-core is stateless and can run with multiple replicas in a cluster
Frontend	Vue 3, Vite, Pinia	Browser UI; talks exclusively to the backend API and is served by the backend

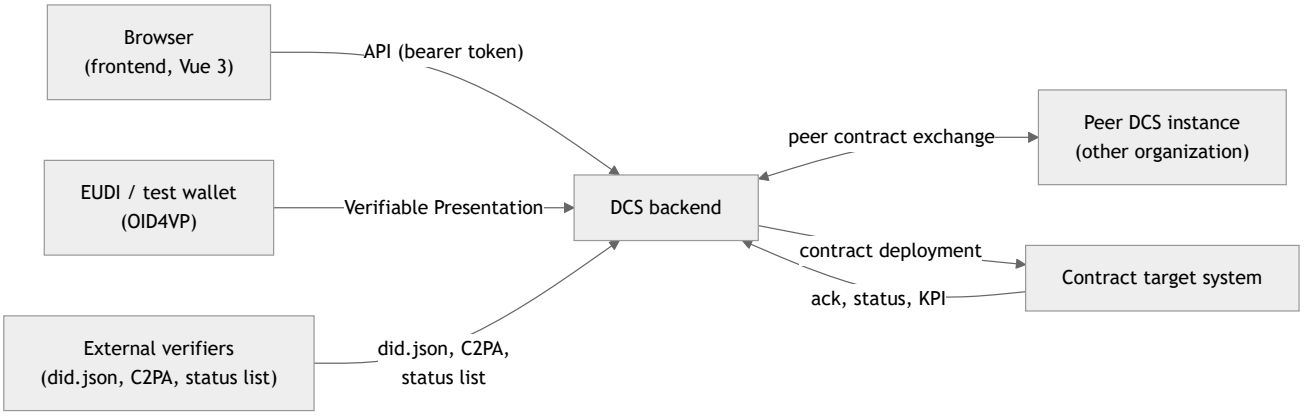
Internally the backend is organized into business domains, each with its own API group; Appendix A lists the endpoints.

Third-party and infrastructure components

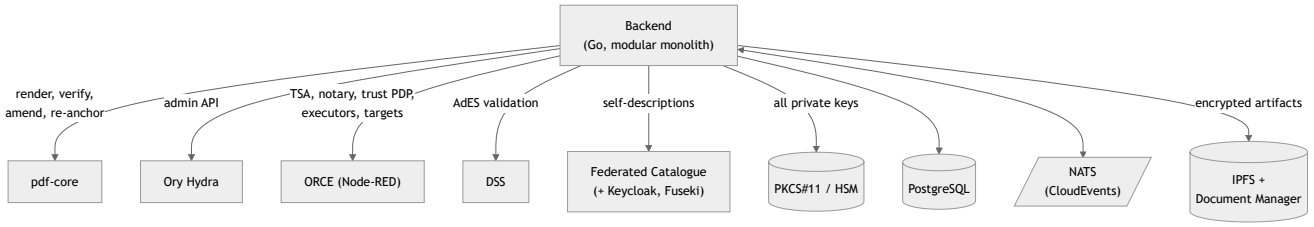
Component	Role in the system
PostgreSQL	Transactional persistence for all backend domains plus separate databases for Hydra and the Federated Catalogue
NATS	Internal event bus (CloudEvents). Carries domain events from the transactional outbox to downstream consumers
IPFS (+ Document Manager)	Content-addressed storage for all artifacts; the Document Manager provides a multi-tenant API in front of it. The backend stores only ciphertexts there
Ory Hydra	OAuth2/OIDC provider of the instance: tokens for frontend users (after OID4VP login) and machine callers
ORCE	XFSC Orchestration Engine (Node-RED): RFC 3161 TSA, archive notary and audit log sink, trust PDP, checkpoint anchor, target-system integration, event webhooks, audit executor, workflow gate execution. ORCE is the extension point for further integration into the XFSC ecosystem; the flows shipped with the DCS are demonstration scope
DSS	EU Digital Signature Service as external AdES validator. Without it the signature submission path accepts no signature
Federated Catalogue (+ Keycloak, Fuseki, its own PostgreSQL)	XFSC catalogue for publishing approved templates as self-descriptions; Keycloak provides the service account tokens
PKCS#11 token (HSM)	Holds all private keys of the instance. Only the backend accesses it
Ingress	External reachability; behind it sit the public resolution paths as well as the authenticated API

Who talks to whom

The external view shows who reaches a DCS instance from outside; the backend is the single point of contact. The browser additionally runs its OIDC flow against the instance's OAuth2 provider (Chapter 05).



The internal view shows the components the backend orchestrates inside the instance:



The most important relationships:

- **Frontend → backend:** the browser talks exclusively to the backend API, authenticated with a Hydra token after the OID4VP wallet login (Chapter 05).
- **Backend ↔ pdf-core:** the backend has the PDF compiled on every relevant change and has incoming documents verified by re-rendering. For the C2PA manifest signature, pdf-core returns the structures to be signed together with the rendered PDF; the backend signs them with the HSM and has them embedded in a second call. pdf-core never holds a signer and never calls back (Chapter 07).
- **Outbox → NATS → consumers:** every business change writes its event into an outbox in the same transaction. A background process publishes on NATS (PDF regeneration, federation dispatch, webhooks, auto-deployment) and anchors audit-visible events as Merkle-committed audit entries (Chapter 09).
- **Backend ↔ peer:** the signed PDF is the wire format of the federation; dispatch happens in `OFFERED`, `NEGOTIATION`, `SIGNED` and `REVOKED`, protected by did:web challenge-response and the fail-closed trust gate (Chapter 08). Failed attempts land in a retry table.
- **Backend ↔ ORCE:** two edge services are hard startup dependencies because they sit in the verification path: the workflow gate executor (Chapter 04) and the audit executor (Chapter 09). As a credential issuer, ORCE additionally runs in its own releases: one per instance for the power of attorney (PoA), one federation-wide as PID issuer (ADR-31).
- **Backend ↔ Federated Catalogue:** a configured catalogue is a hard startup dependency: the backend checks it functionally at startup and synchronizes the schemas.
- **Status lists:** every issuer serves and signs the revocation list of its own credentials; an unsigned list is accepted nowhere (ADR-34). The issuer releases serve the login, PoA and PID credentials, the backend the self-issued lifecycle and signing summary credentials (Chapter 07).
- **Backend ↔ target system:** handover after signing completes; the confirmation arrives asynchronously as a callback from the target system, which authenticates as its own OAuth2 client (Chapter 04).
- **Public surfaces:** reachable without authentication are the DID document, the agreement credential with its rules document, the C2PA manifest store, the status list and the Semantic Hub resolution (Appendix A).

Trust and key architecture

The keys in the PKCS#11 token are separated by purpose: instance DID, VC issuance, OID4VP request signing, C2PA manifest signing and key agreement for artifact encryption (Chapter 05). pdf-core, the frontend and all third-party components are keyless with respect to the instance identity. The instance deliberately holds no contract signing key; the signer holds it (Chapter 06).

Trust verification towards peers rests on independent, fail-closed layers: the certificate chain of the peer DID, a challenge-response signature per request, the trust gate of agreement credential and local policy endpoint (ADR-19), and the check of the power of attorney behind every counterparty signature (ADR-31, Chapter 08).

Operational view

The components of an instance are deployed via a shared Helm chart. The credential issuers are separate releases of the same ORCE chart, installed before the instance: their root certificates and status lists outlive a re-installation of the DCS release, and the trust anchors are read from what an issuer already serves. Two complete instances can run in parallel. The backend checks required external dependencies at startup and fails hard instead of running degraded (Chapter 10).

03 Templates and Contract Content

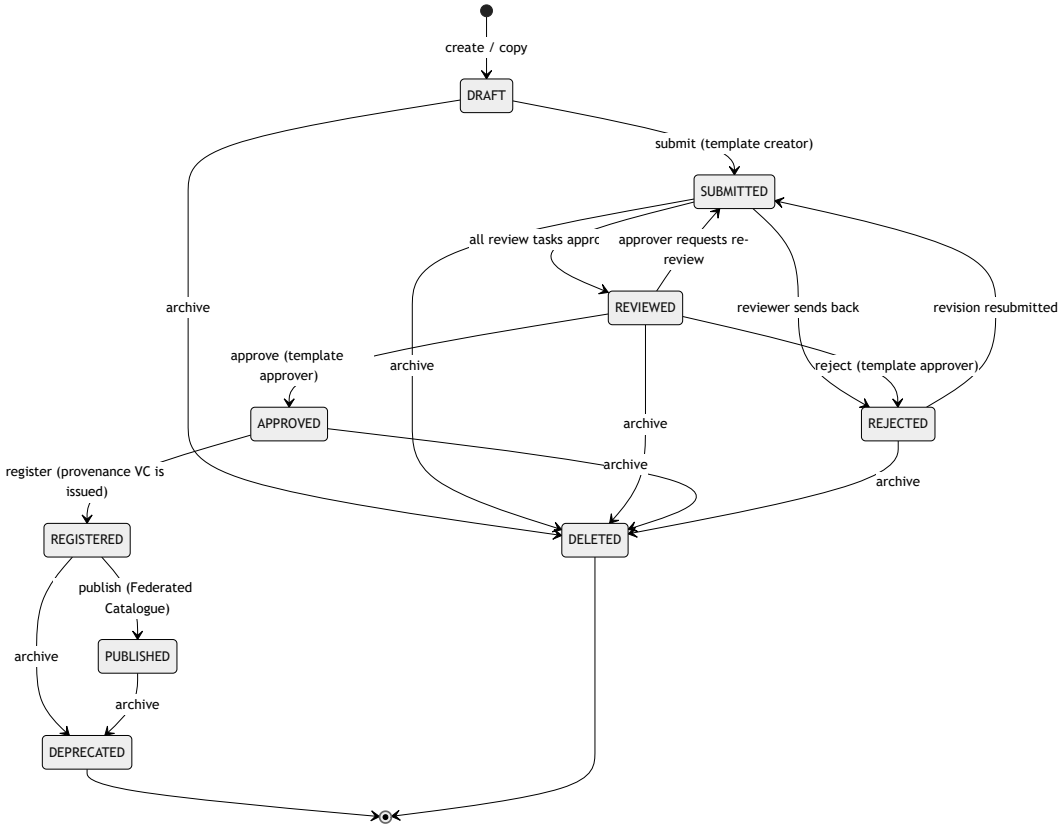
This chapter describes how contract templates are created, reviewed, versioned and published, and how the Semantic Hub defines the machine-readable meaning of all content: JSON-LD context, SHACL shapes, domain field catalog, ODRL profile and clause catalog.

3.1 Components involved

Component	Responsibility
Template Repository	Template lifecycle, review/approval tasks, versioning, provenance
Semantic Hub	Versioned store and delivery point for all semantic artifacts
Federated Catalogue (XFSC FC)	External catalogue to which registered templates are published as self-descriptions
Template Catalogue Integration	Read access to the catalogue; this is how other instances obtain published templates
pdf-core	Deterministic rendering of the documents (Chapter 07)

3.2 Template lifecycle

A template passes through a multi-stage approval process with a four-eyes principle before it becomes usable for contract creation and visible in the Federated Catalogue.



In addition to the diagram:

- Every write operation checks the supplied timestamp against the current state of the template; a stale state is rejected with a request to reload (lost-update protection).
- A reviewer must semantically verify the template before approving it. Role check and task responsibility are two separate hurdles: the RBAC role permits the operation, the task binds it to the specific template.
- Registration fixes one version as the publishable one; the instance issues a provenance Verifiable Credential over exactly this version.
- On publish, the DCS instance (its `did:web`) acts as the issuer, not the clicking employee (ADR-18). The catalogue call runs outside the database transaction; a duplicate reported by the catalogue counts as success, so a repeated publish after a partial failure is idempotent.
- `archive` acts depending on state: registered or published templates become `DEPRECATED` (they remain referenceable); all earlier states become `DELETED`.

The lifecycle events reach registered external subscribers via the webhook platform (Appendix A).

Versioning by copy. Templates are not versioned in place. The copy command duplicates a template under a new DID: an unregistered source starts a new version line; a registered or published one yields the next version of the same line. Every new version passes through the full approval process again.

Role	May
Template Creator	Create, edit, copy and submit templates
Template Reviewer	Verify and review submitted templates
Template Approver	Approve or reject reviewed templates
Template Manager	Overarching management rights; the only role holder allowed to register Semantic Hub versions

Templates are not synchronized directly between instances; the exchange path is publication to the catalogue and read access by other instances via the catalogue integration. The catalogue hands over content, never authority: an importing instance registers the found template as its own new template under its own DID in `DRAFT` and runs it through its own approval process. The ODRL semantics travel unchanged and bind the importing instance just like the author: the same rules, range limits and consequences are enforced on both instances with the same result.

3.3 Semantic Hub

The Semantic Hub is the versioned store for everything the DCS creates and validates its documents against. Each entry is a pair of name and kind (`kind`) with an immutable version history and exactly one active version. The base documents shipped with the backend:

Hub entry (name / kind)	Content and effect
`facis-dcs` / context	The JSON-LD context every document anchors as `@context`; its prefix-to-IRI mappings are enforced on every artifact
`facis-dcs` / shapes	SHACL shapes of the canonical document envelope, the blocking validation graph
`clause-catalog` / shapes	Typed clause node shapes; at the same time the declaration of their terms (3.4)
`facis-sla` / ontology	The domain field catalog: `dcs:DomainField` entries with value ranges and taxonomy options; parsed RDF configuration
`dcs-odrl-profile` / ontology	The ODRL profile: constraint operators, actions, default action. Drives the condition editor
`facis.sla.basic` / profile	Domain validation rules at statement level; shipped as demonstration content
arbitrary shape libraries / shapes	External SHACL vocabularies registered via the hub (e.g. Gaia-X classes): building blocks for typed business objects in `dcs:contractData`

How it takes effect:

- **Seed at startup.** The base documents are installed at startup; a document that differs from the last known state becomes the next active version. Versions are immutable. If the seed fails, the instance does not start.
- **Pinning rather than drifting (ADR-8).** A newly created document anchors the active version via version-fixed URLs. A later activation only changes what new documents are validated against; existing ones remain bound to their version and stay reproducibly verifiable. An explicit rollback exists for corrections.
- **Public resolution.** The resolution endpoints are unauthenticated so that external verifiers can dereference the anchors embedded in a document without a DCS account.
- **Hermetic resolution.** A document may reference external context IRIs only if they are registered in the hub under their original IRI; validation runs exclusively against the hub holdings, never over the network. An unregistered IRI is rejected at creation.

What is checked where

Layer	Subject	Effect
SHACL against the pinned shapes graph	Structure, types, cardinalities of the envelope; value constraints of the clause catalog and all active shape libraries	**Blocking** at offer, submission and signing preparation
ODRL constraint evaluation	The values carried inline in the contract against its own ODRL terms	**Blocking** at approval and signing preparation (Chapter 09)
Completion check	Unfilled mandatory fields and empty values referenced by ODRL rules	**Blocking** at offer, approval and signing preparation
Validation profile (statement rules)	Domain rules over statements in the document	Feeds into the workflow gate: "error" blocks, a warning puts the transition under manual review (Chapter 09)
Value constraints of the domain field catalog	Permissible values of a domain field	**Not enforced server-side**; they drive the editor and client-side checks. To enforce a value limit, express it as SHACL

At signing preparation, SHACL evidence (shapes version and a hash over the findings list) is embedded into the signing summary credential so that external verifiers can repeat the validation against exactly the pinned shapes.

3.4 Clause catalog

The clause catalog is an independently versioned shapes entry in the hub: typed clause node shapes with target classes, datatypes, value lists, range limits and patterns. It serves two consumers from one source: the clause endpoint delivers the palette of clause types to the template builder, with the forms generated client-side from the raw Turtle (ADR-10); submitted clause instances are validated server-side against the same shapes graph. Palette and validation therefore cannot drift apart. The catalog is at the same time the declaration of its terms; a separate vocabulary does not exist.

3.5 The canonical document envelope and the contract field model

Every DCS document, template and contract alike, is a single canonical JSON-LD envelope (`dcs:ContractTemplate` or `dcs:Contract`):

Level	Content
`dcs:metadata`	Title, description, template type
`dcs:documentStructure`	Human-readable structure: ordered blocks (`dcs:Section` , `dcs:TextBlock` , `dcs:Clause`) and a layout tree
`dcs:contractFields`	Flat list of field declarations (`dcs:ContractField`)
`dcs:contractData`	Generic graph of typed business objects; properties carry literals, field references or object references
`dcs:policies`	Exactly one enclosing ODRL policy
`dcs:signatureFields`	The declared signature fields with their required signature level; the level is pinned at preparation and enforced when a signature is submitted, a valid AES on a QES field is refused (Chapter 06)

The contract field model (ADR-22, extended by ADR-23) separates field declaration from business data: each `dcs:ContractField` declares `@id` , label, datatype and a mandatory flag. In the data graph, a property carries exactly one of three things: a literal, a reference to a declared field (a negotiable leaf), or a reference to another business object. Every reference must resolve within the document itself; embedded blank nodes are not permitted. Clause prose, layout and ODRL operands reference the same field IDs. The document is therefore self-contained: authoring, validation, rendering and policy evaluation resolve a field via its `@id` without consulting a template snapshot.

Open fields and the completion gate. On a template, a field may be declared but valueless; it is filled at contract creation and during negotiation. Offer, approval and signing preparation reject a contract with unfilled mandatory fields or with empty values referenced by ODRL rules. No party can sign a document with open terms (Chapter 04).

Hierarchy. A contract may reference exactly one parent contract; the link points exclusively from child to parent (ADR-7). Backward references are rejected during normalization because they could point to documents the reader is not allowed to see.

Shape libraries drive editor and validation. Registered SHACL libraries join the active validation graph; the editor derives its palette of typed data objects from them. Validation runs against a field-materialized copy of the document: references to filled fields are dereferenced to their value, so ordinary libraries written against instance data constrain the document directly. References to unfilled fields are absent from the copy; the cardinality rules then name what the negotiation still has to deliver (ADR-23).

ODRL rules. The policy is an `odrl:Offer` until the first signature and is sealed into an `odrl:Agreement` by the signature. Each rule carries action, assigner, assignee, target and the human-readable clause it operationalizes; Chapter 07 describes the independent cross-check of both representations.

At runtime there is exactly **one** internal form, the canonical `dcs:` -prefixed envelope. It is enforced at the ingestion edges: a document whose `@context` remaps a hub prefix to a different IRI is rejected. A document received from a peer instance already arrives in this form because the PDF carries exactly these bytes as its attachment (Chapter 07).

3.6 Operational behaviour

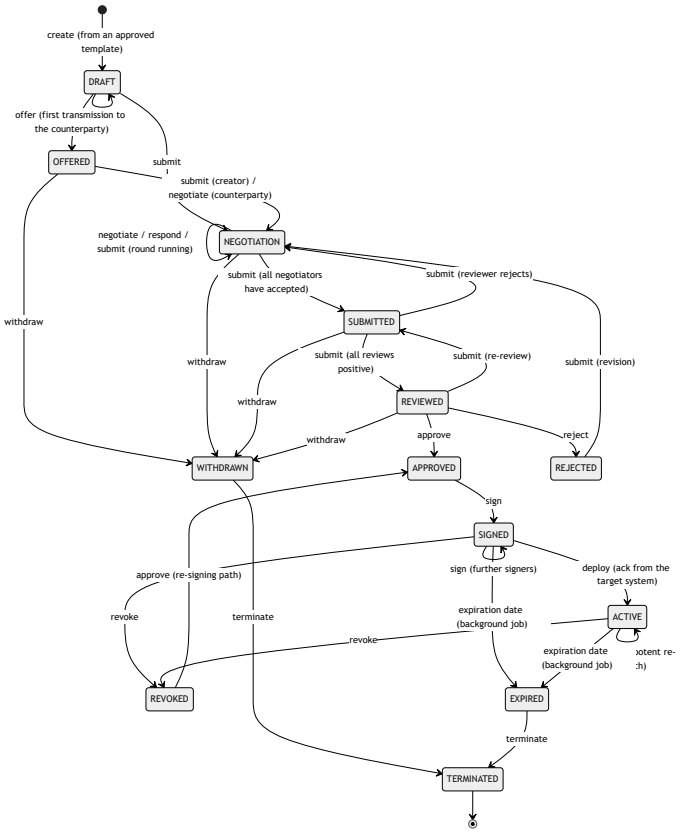
- Without a configured Federated Catalogue, `publish` fails with an unambiguous configuration error; all upstream steps work independently of it. A configured catalogue is a hard startup dependency (Chapter 10).
- If the local state change fails after a successful catalogue publication, the next publish call heals the state.
- Concurrent editing is detected via the change timestamp and answered as a request error; no silent overwrite takes place.
- Every lifecycle step creates its audit event in the same transaction as the state change (Chapter 09).

04 Contract Lifecycle

This chapter describes the lifecycle of a contract from creation out of an approved template through negotiation, review and approval to signing, activation, termination and erasure: the central state machine, the permission model of roles and tasks, the workflow gates, negotiation, and the difference between a local and a cross-instance contract.

4.1 The state machine

There is exactly one contract state machine in the entire backend (ADR-2). Permissible transitions are stored as an explicit table; every command handler validates against it before executing its business logic. An illegal transition is rejected as a request error with a descriptive message.



In addition to the diagram:

- **Terminate** is reachable from every non-terminal state (shown only by example in the diagram).
- **Submit is deliberately overloaded:** its effect depends on the state (4.4). The transition table bounds the legal outcomes; which one occurs is decided by the task orchestration.
- **Withdraw** is possible only until approval.
- **Expired** is set by a background job based on the expiration date, from every running state from offer to `ACTIVE` (shown from `SIGNED` by example in the diagram); drafts as well as rejected, withdrawn and revoked contracts are left untouched. A database view already shows the state at query time before that. With the transition, `contract.expired` is published.

- **Renew** is not a state transition: the renewal creates a new, linked contract instance in `DRAFT` with a back-reference to the original. The signers of the previous period are removed from the new draft.

Extrinsic lifecycle. Across the instance boundary, every contract presents a derived negotiation lifecycle: all formation states up to `REVIEWED` as `proposed`, mutual internal approval as `agreed` (this is where the signing gate opens), a contract with all signatures present as `executed`. For `executed`, every declared signature must actually exist, including the cryptographically verified signature of the counterparty. Both instances derive the same value from the shared artifact; no state is synchronized across the federation boundary.

4.2 Roles, tasks and RBAC

Authorization has two separate layers:

1. **Local RBAC roles** determine which user may perform an operation at all: Contract Creator, Reviewer, Approver, Negotiator, Manager, Signer and Observer. Machine callers appear in their own `Sys.` class. There is no dedicated machine negotiator or observer role, but the negotiation and read endpoints accept the existing `Sys.` scopes (Sys. Contract Creator and Sys. Contract Reviewer for negotiate and respond), so machine callers negotiate and read contracts through those. The machine signing class carries no signing permission (Chapters 05 and 06). Audit read access is held by Auditor, Archive Manager and Compliance Officer.
2. **Tasks bind responsibility to the specific contract.** The role permits the operation; the task decides whether this caller is responsible for this contract.

The tasks are small sub-state-machines that steer the large transitions as fan-in gates:

Task	States	Gate
Negotiation task	open → accepted	Only when no negotiator task remains open can `NEGOTIATION → SUBMITTED` happen
Review task	open → approved / rejected	Steers `SUBMITTED → REVIEWED`; a rejection reopens the negotiation
Approval task	open → approved / rejected	Steers `REVIEWED → APPROVED`

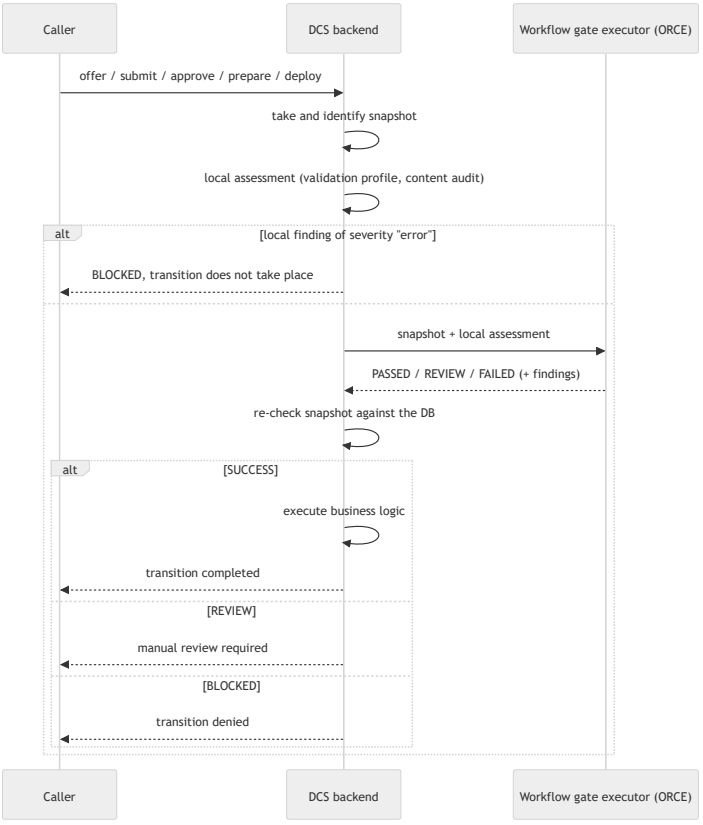
Each instance creates and owns exclusively its own tasks; task states never cross the instance boundary (4.5).

The lifecycle is API-complete. Every step from creation through review and approval to signing and archiving can be triggered individually via the authenticated HTTP API and produces the same state and evidence chains as operation via the user interface; the frontend is just another API client. Machine callers can thus drive contracts fully system-controlled (Chapter 05). Only the signature remains a personal wallet ceremony, which also runs via the API without a user interface (Chapter 06).

For read access an additional party check applies: the canonical contract document is readable only for authorized parties. A denial is recorded as an audit artifact before the rejection is returned; this feeds the compliance risk "unauthorized access" (Chapter 09).

4.3 Workflow gates: the externalized check before the transition

Five transitions first pass through a **workflow gate**: offer, submission, approval, signing preparation and deployment. A gate works on an immutable snapshot of the contract (version, state, content hash, pinned shapes, profile version):



- **The run is evidence.** Every gate run is persisted with correlation ID, snapshot identity, request and response, and is retrievable via the compliance API. Two runs of the same snapshot at the same gate are the same run.
- **The snapshot is re-checked after the response.** An intervening content change discards the result; purely technical writes (PDF regeneration, audit) do not block.
- **`REVIEW` is a holding state, not an error.** A compliance officer decides the run with a justification; on approval, exactly the halted transition is resumed.
- **Fail-closed.** An unconfigured or unreachable executor blocks; the run is recorded anyway with its cause. The executor is therefore a hard startup dependency.

4.4 Negotiation

Negotiation takes place in `NEGOTIATION`, or on a received offer in `OFFERED`, and revolves around change requests:

- **Propose.** A change request is either free text (a note only for the negotiation audit trail) or a structured redline on the contract data. A redline is applied to the contract data immediately; the PDF re-renders with the proposed value and is shipped to the counterparty (4.5), which reviews the redline in the received document.

- **Decide.** Every responsible negotiator accepts or rejects a change request (rejection with a justification).
- **Close the round.** Submit closes the round: if negotiator tasks are still open, the contract stays in `NEGOTIATION`; if accepted change requests are to be merged, they are merged, the version increments, and a new round begins; if nothing remains to merge, the contract moves to `SUBMITTED`.

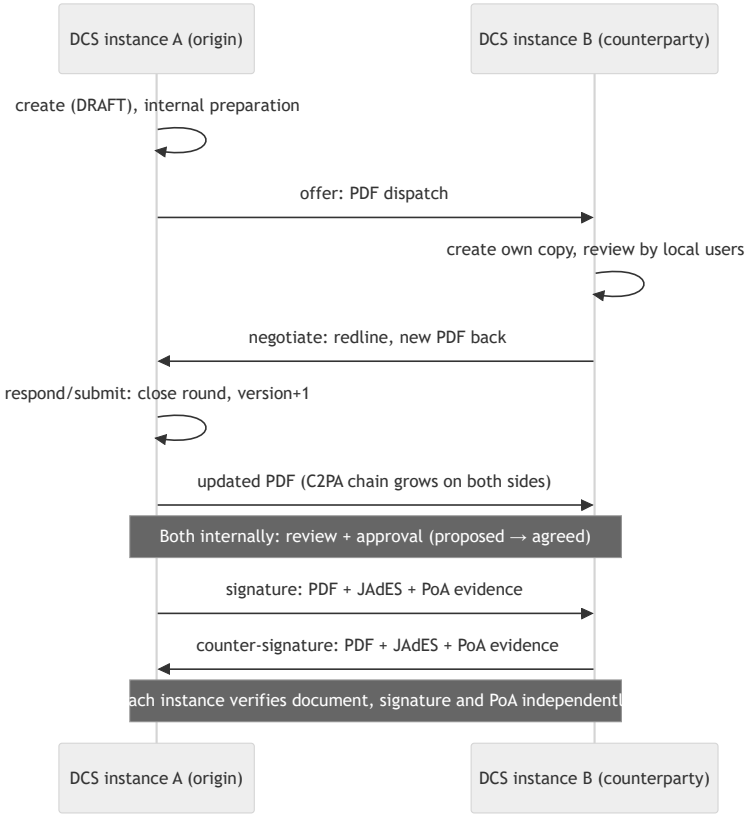
Negotiation drafts. A negotiator can prepare a change request: the draft is stored per (contract, party) and shared by all negotiators of the same party. It changes no state, creates no audit event and does not leave the instance; only proposing turns it into a change request and deletes it.

4.5 Local vs. cross-instance

In this version of the DCS, a contract has exactly one **origin instance** and exactly one **counterparty**, a peer DCS identified via `did:web` and named at contract creation. Locally (origin equals the calling instance), all responsibilities are local tasks, and nothing leaves the instance except an optional deployment.

Across the boundary, **each instance runs the state machine on its own copy**; calls are not forwarded to the origin. Only the contract document crosses the boundary (ADR-13). Authorization mirrors this: on the origin instance, negotiating requires a local negotiator task; on the receiving side, the right derives from being the named counterparty. Editing the draft (update) is permitted only on the origin instance.

The PDF is the wire format. At every dispatch-relevant step the contract PDF is shipped: with embedded JSON-LD, C2PA provenance chain and any existing signatures. Chapter 08 describes how the receiver authenticates the sender and verifies the document.



Chapter 08 covers the trust model and peer authentication, Chapter 06 the signing ceremony.

4.6 After signing: deployment, revocation, termination

Target system registry and designation (ADR-25)

Contract target systems are an administered registry of the instance with name, endpoint and active flag. Every contract **designates** its deployment target. A deactivated target cannot be designated; a designated target cannot be deleted. A contract without a designated target does not deploy, and that is a normal outcome: the automatic trigger after signing completion then does nothing, and a manual deploy is rejected with a reason.

Deployment

A fully signed contract is transmitted to the designated target as a JSON-LD payload with contract DID, version, content hash, timestamp and the ODRL policy; a manual deploy may go to a different registered target via an override without changing the designation.

The **deploy gate** requires every declared signature field to be signed. For the instance's own fields the locally recorded signature counts, for the counterparty the verified JAdES artifact from its signed dispatch. One party cannot bring a contract into execution alone. The instance holds exactly one counterparty signature per contract; a contract with three or more parties is therefore not deployed, instead of treating an incomplete signature situation as complete.

The deployment record captures the endpoint as it was at dispatch; the dispatch carries a correlation ID and a recomputable content hash. A failed dispatch is recorded on the record and reported as a risk by the compliance monitor. The confirmation arrives asynchronously as a callback and switches the contract to `ACTIVE`; the target system authenticates as its own OAuth2 client and can only acknowledge its own deployments (ADR-27). The acknowledged execution evidence receives an RFC 3161 timestamp and is attached to the archive entry. Via the same callback, the target system later reports KPI observations together with its verdict on the respective ODRL rule (Chapter 09).

Revocation, termination, expiry

- **Revocation** moves the contract from `SIGNED` / `ACTIVE` to `REVOKED`; a renewed approve leads back to `APPROVED` (re-signing). For a cross-instance contract, the revocation goes immediately to the counterparty and is the only peer state the receiver adopts (Chapter 08).
- **Terminate** with a reason ends the contract for good.
- **Expiry:** the background job persists `EXPIRED` and publishes the event. The stored expiration policy is **recorded but not executed automatically**; the follow-up action is manual or orchestrated via webhooks. A contract without an expiration policy is skipped by the job and logged.

Archive and erasure

Concluded contracts reside with their evidence in the long-term archive, structurally searchable (full text, name, state, party, period, tags), with a dashboard for inventory and compliance statistics.

Deleting an archive entry requires a justification and has two effects: the entry is marked deleted and remains provable, the archived snapshots are removed; and the **contract's content keys are destroyed** (ADR-28), locally and by request to every counterparty instance. Afterwards export, verification and bundle retrieval are permanently impossible; list and metadata views keep working. An unreachable counterparty does not block: the request stays open and is redelivered periodically. Progress is visible via a dedicated status query (Chapter 09).

4.7 Operational behaviour

- **Optimistic concurrency:** state-changing endpoints require the timestamp of the state seen; a stale state is rejected with "please reload", for synchronized contracts with "force synchronization and reload". The comparison uses the content timestamp, so background writes cause no false alarms.
- **Illegal transitions** are caught by the transition table and rejected as request errors with a descriptive message.
- **Blocking checks:** SHACL error findings and unfilled mandatory fields or ODRL-referenced empty values prevent offer, approval and signing preparation (Chapter 03).
- **Every state change** writes its audit event in the same transaction; the audit trail is gapless with respect to the state history (Chapter 09).
- **Lifecycle events** reach external subscribers via the webhook platform with a delivery log and acknowledgement (Appendix A).
- **Deployment operations** can be correlated with their callbacks via correlation IDs; a dispatch that was never delivered is distinguishable from an acknowledged one.

05 Identity and Authentication

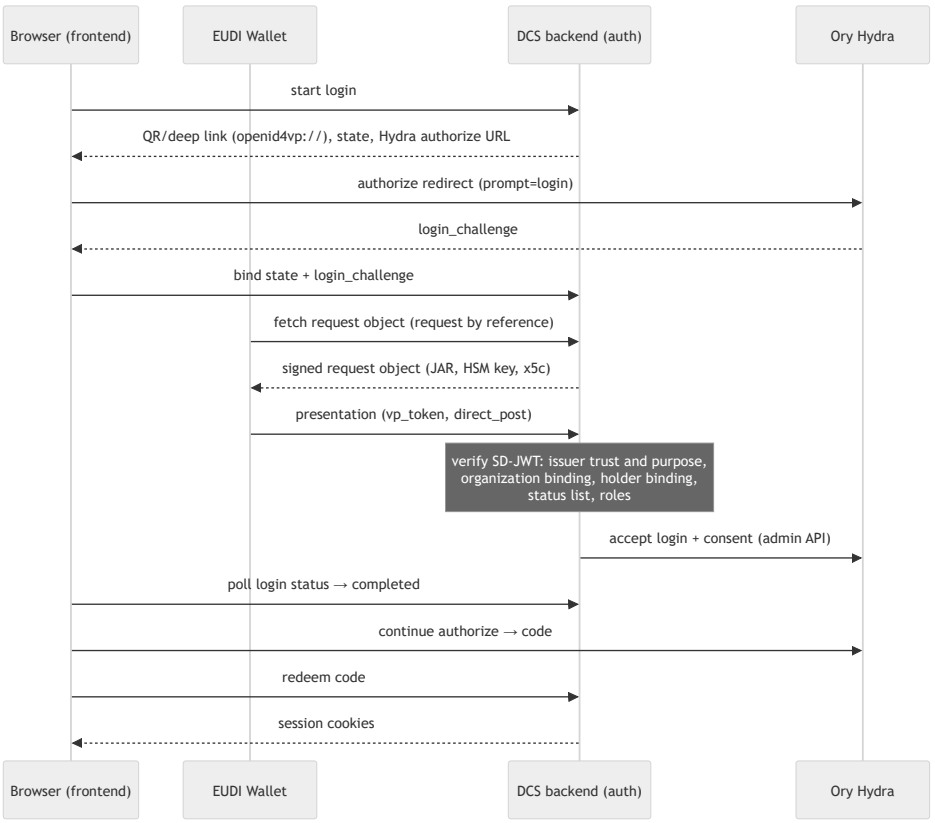
The DCS has three strictly separated identity planes: **end users** (people with a wallet), **machine identities** (automated callers with their own OAuth2 client) and the **DCS instance itself** (a `did:web` identity with HSM-backed key material). Across all three sits the question of which credential issuers the instance trusts, and for what purpose (5.4).

5.1 End users: login via OID4VP and SD-JWT VC

Component	Role
EUDI Wallet (or compatible wallet)	Holds the user's credentials and presents them via OpenID4VP
DCS backend (auth domain)	OID4VP verifier: builds the presentation request, verifies the presentation, decides the login
Ory Hydra	OAuth2/OIDC provider: issues the tokens, manages the browser session
Frontend	Shows QR code / deep link, polls the login status, drives the OIDC flow

Hydra knows no users and no passwords. The authentication decision is made by the DCS backend based on the verified wallet presentation; Hydra is the token factory whose login and consent challenges the backend accepts via the admin API, carrying the verified claims (subject, organization, roles) into the session.

The credential: Proof of Authority (PoA). The login requires a credential of type `urn:dcs:poa:v1` in format `dc+sd-jwt` with two decisive claims: `organization`, the party the user acts for, and `roles`, the roles granted to them. A DCQL query describes the requested credentials; the default query requires holder binding and can be overridden by configuration. The `organization` is a party identifier, not the identity of the deployment: one instance legitimately serves several organizations.



The request object (JAR) is signed with a dedicated HSM key and carries the instance's certificate chain in its header; the client identifier presented to the wallet follows the `x509_san_dns` scheme and matches the SAN of the leaf certificate. Login states are time-limited and renewable.

Verification of the presentation. Every stage is a hard abort criterion:

1. **Issuer and purpose:** the issuer must be in the trust configuration and admitted for `login`; its key is resolved via the mechanism declared for it (5.4). The credential type must be admitted.
2. **Organization binding:** the disclosed `organization` must be among the organizations this issuer may speak for.
3. **Holder binding:** subject and key-binding JWT must be bound to the credential's binding key as well as to audience, nonce and disclosure hash of the ongoing presentation (replay protection).
4. **Status list:** revocation check on every path, for every purpose; the status list itself must also be signed and trustworthy. An unreachable status list leads to rejection.
5. **Roles:** every disclosed role must be a valid DCS role; without roles there is no login. The roles end up in the access token and are the basis of the RBAC check (Chapter 04).

PID presentation. In addition there is a sessionless PID presentation flow for proving the natural person, in particular as a precursor to the signing ceremony (Chapter 06). The verification chain is the same with purpose `pid`; the organization binding does not apply. A PID issuer is by construction a third party: the instance must not itself issue the identity proof it later accepts as evidence (ADR-31).

Protection mechanisms:

- **IP lockout:** after five failed token validations within the time window, the source IP is blocked for 15 minutes. A valid login with a missing role does not count towards the lockout; it is audited as an authenticated access event.

- **Audit trail:** access and presentation attempts are persisted and fed into the audit trail (Chapter 09).
- **Per-credential request budget:** above a one-minute budget the DCS responds with `429` and `Retry-After`. Unauthenticated routes do not count; counting uses a truncated hash of the credential, never the raw token.

5.2 Machine identities: issued credentials via Hydra

Machine callers authenticate with the client credentials grant at Hydra. Every caller receives its **own OAuth2 client**, provisioned by the DCS via the Hydra admin API (ADR-27). A **machine identity registry** records, per identity, the client, the party its actions are attributed to, the roles and the active flag. Permissions are read from the registry by client ID on every request; without an active registry entry no machine caller is accepted.

The credentials are **issued, not configured**: the client secret appears exactly once in the response; the DCS never stores it, Hydra keeps only a hash. Rotation invalidates the old secret immediately, deletion also removes the client, deactivation takes effect on the next request; all of this is managed as audited operational action by the Sys. Administrator. The deployment configuration `DCS_SYSTEM_CLIENTS` is a **seed** for callers that must exist before the first login; it exists primarily for CI and test environments. At runtime only the registry is read, and an invalid role in the seed prevents startup.

Contract target systems are not general machine identities: their role authorizes only the deployment callback, and there only deployments to exactly this target. The callback credential follows the same mechanism (Chapter 04).

Role boundary human/machine. The `Sys.` roles deliberately do not mirror the human ones one to one. There is no dedicated machine negotiator or observer role; machine callers still negotiate and read contracts through their existing `Sys.` scopes, because the negotiation endpoints accept Sys. Contract Creator and Sys. Contract Reviewer and the read endpoints accept every contract-facing `Sys.` role (Chapter 04). What machines cannot do: sign (machine signing is not a person's AES, ADR-17), access the catalogue, or register new Semantic Hub versions; those remain reserved for human roles.

5.3 Instance identity: did:web with HSM keys

Every instance owns a `did:web` identity; the DID document is served under `/.well-known/did.json`. Resolution follows the did:web method in full, including identifiers with path segments, so several instances can share one host (Chapter 08).

The private keys reside **exclusively** in the PKCS#11 token and never leave it (ADR-1). There is no software fallback: with a wrong module path, token label or PIN the process does not become healthy. A missing token is distinct from that: the process waits for it, because on a fresh installation the provisioning may still be running (Chapter 10).

The keys are separated by purpose (all ECDSA P-256):

Default label	Purpose
`dcs-did`	Instance identity: JAdES signatures of the federation, DID challenge-response
`dcs-vc`	Lifecycle, provenance and federation agreement credentials
`dcs-oid4vp-jar`	Signing the OID4VP request objects
`dcs-c2pa`	COSE signatures of the C2PA manifests and signature of the instance's own status list (Chapter 07)
`dcs-ecdh`	Key agreement: unwraps the content keys of artifact encryption (ADR-28)

The key agreement key is a hard startup dependency: at startup the instance wraps a test key against the `keyAgreement` entry published in the DID document and unwraps it with the HSM. If this fails, no stored artifact would be readable, and the instance does not start.

The instance deliberately holds no contract signing key: contract signatures are created exclusively by the signer with their own key (ADR-12/ADR-20, Chapter 06). pdf-core never holds a signer; it only returns data to be signed.

Key rotation uses versioned labels; old versions remain in the token so historical signatures stay verifiable. The key inventory can be queried by the Sys. Administrator; the rotation itself is an operational procedure (key management concept, deployment guide).

5.4 Issuer trust: purpose-scoped, organization-scoped, mechanism-open

Whether a credential is accepted is decided by three separate questions (ADR-31):

Purpose. Every issuer is admitted for an explicit set of purposes; an entry without a purpose is rejected at load time. Which issuers receive which purpose is an operator decision.

Purpose	Meaning
`login`	Its credentials may establish a session on this instance
`peer`	Its credentials are accepted in a signing ceremony, and its attestation of a counterparty's power of attorney is verified
`pid`	It may attest the identity of a natural person

Organization. An issuer may only attest organizations listed in its own entry; a missing list never means "any". PID issuers are exempt, because an identity proof attests a person, not an organization.

Mechanism. Every issuer declares how its verification key is resolved:

Mechanism	Resolution
`jwks`	Keys stored in the entry, matched via the key ID
`x5c`	Certificate chain in the credential header, verified against configured trust anchors
`did:jwk`	Key decoded from the issuer identifier
`did:web`	Key from the issuer's DID document
`orce`	Delegated to a configured ORCE flow

A non-resolvable mechanism is rejected **at load time**: a deployment learns about an unsupported trust configuration at startup, not when the first wallet arrives. The `x5c` path is strict: without configured trust anchors a credential with a certificate chain is rejected, and the leaf certificate must name the issuer and be issued for this purpose, so an ordinary TLS certificate of the same host cannot sign credentials.

Trust anchors in this local configuration. Whether an admitted issuer is in turn legitimized by a superior authority, for example via a chain check up to an ecosystem anchor, is not checked by the DCS; this legitimization chain is deliberately deferred.

Above the trust configuration sits an **authorization policy** as its own artifact. A policy that cannot be compiled stops startup; one that cannot be evaluated at runtime denies. It ranks above the trust document; the startup attestation therefore covers both (Chapter 09).

5.5 Federation trust model at a glance

Whether a foreign instance is accepted as a peer is decided by the **Federation Trust Gate** (ADR-19), consulted on dispatch and receipt: the peer's agreement credential plus the local policy decision point as the sole authorization authority. The gate is fail-closed; every rejection is recorded as an incident. Chapter 08 describes the full flow.

06 Signatures

This chapter describes how a contract is signed electronically: the wallet-driven signing ceremony, the signature levels, the formats PAdES (human-readable PDF) and JAdES (machine-readable JSON-LD), validation via the EU Digital Signature Service (DSS) and timestamping.

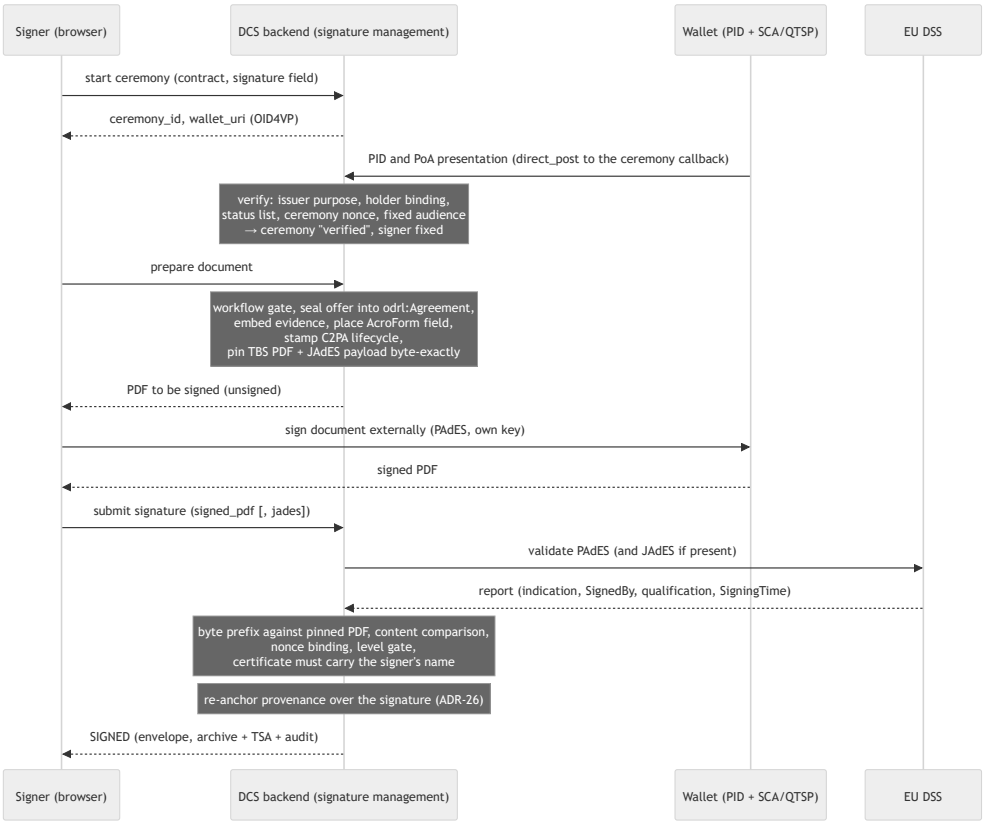
The governing principle (ADR-12, ADR-3, ADR-20): **the DCS never signs contracts itself and holds no contract signing key.** It prepares the document, the signer signs externally with their own key, and the DCS validates and finalizes the result. Only this satisfies the signer's sole control over their key (eIDAS Art. 26). What the DCS accepts must build byte-exactly on the document it prepared itself, be bound to the ceremony, and carry a certificate naming the verified signer.

6.1 Components involved

Component	Role
Signature management (backend)	Ceremony management, document preparation, acceptance and validation of external signatures, compliance view
Signer's wallet + SCA/QTSP	Presents the identity credential, fetches the document, creates the PAdES signature with the signer's key
pdf-core	Deterministic rendering of the base PDF, content comparison, provenance re-anchor; never holds a signer (Chapter 07)
EU DSS	External AdES validator (ETSI EN 319 102-1) for submitted PAdES and JAdES signatures
ORCE + TSA	RFC 3161 timestamp for the archive entry; the backend verifies the response against the stored TSA certificate
Frontend (signing list, Secure Contract Viewer, Signature Compliance Viewer)	Guides the signer through the ceremony; shows signature status, integrity and DSS findings

6.2 The signing ceremony

Only someone who has previously proven their identity as a natural person in a **ceremony** may sign: an OID4VP presentation, bound to the contract and a declared signature field. Without a completed ceremony the DCS rejects any document preparation and signature acceptance.



In addition to the diagram:

- The ceremony status can be polled (`pending` , `verified` , `expired` , `failed`); the callback is authorized via the unguessable ceremony ID, and a presentation against an expired ceremony is rejected. Organization and roles of the power-of-attorney credential are retained as signature evidence, because the counterparty verifies them later (Chapter 08).
- The preparation embeds the signature evidence **inside** the byte range that is signed later (embed-then-sign, ADR-3) and **pins** the exact bytes to be signed (TBS PDF and canonical JAdES payload) to the ceremony (ADR-20). All side effects happen exactly once, here.
- The submission is a pure validation and recording step (6.4). Finalization re-anchors the provenance, creates the archive entry with notarization and timestamp, and marks the ceremony as consumed atomically, so two concurrent submissions cannot both finalize.

Signature and provenance re-anchor (ADR-26). C2PA hard binding and PAdES signature both want to be "the last thing in the document". The DCS resolves the conflict in a fixed order: the lifecycle manifest before the signature (the signature covers the provenance), and after the validated submission a provenance-only update manifest as an incremental update over the signed bytes. The signature remains untouched and continues to verify. The accepted price: PDF readers and DSS report the document as "changed after signing". That is accurate, the only permissible cause is this re-anchor, and the DCS's own verification proves exactly that (Chapter 07).

Multi-signer. If a contract declares several signature fields, a verified ceremony must exist for each field before the first signature is applied: the evidence of all signers must be in the PDF before a PAdES signature freezes it. An already signed field cannot be signed again.

QR/deep-link variant (OID4VP document retrieval). For fully wallet-driven signing, the ceremony publishes a signed request object following the EUDI "walletdriven-signer" profile with two documents, the PDF to be signed and the pinned JSON-LD payload, so one ceremony delivers PAdES and JAdES over the same content. The wallet operates its own SCA/QTSP leg and posts the signed documents via `direct_post` to the callback, which runs the same validation path; the response must bind the ceremony's fresh request nonce (6.4).

6.3 Signature levels and formats

The required level is a property of the contract, not of the caller. Each signature field declares it in the document; if absent, AES applies. The level is pinned at preparation and enforced at submission against the level **actually achieved**: a valid AES on a field requiring QES is rejected, and the achieved level is recorded (ADR-20). Comparison uses the scale SES < AES < QES; to enforce a requirement, demand AES or QES.

v1 scope of delivery (ADR-24). The architecture is not AES-limited; the achieved level is determined by credential and trust service. With the test-grade v1 providers (project-owned CA, test wallet, demonstration TSA, DSS test instance) the result is a wallet-based AES. QES is excluded from the v1 deliverable; contracts subject to a statutory written-form requirement cannot be demonstrated in v1. The bundled example PID issuer performs no identity verification of the person; its credential is not a EUDI PID and must not be treated as one (ADR-31).

The two formats:

- **PAdES** is the signer's signature over the human-readable PDF, visible in the AcroForm field, created externally.
- **JAdES** (ETSI TS 119 182-1, baseline B) is the counterpart over the machine-readable representation: a compact JWS over the canonicalized form of contract DID, version and full JSON-LD document. In a published wallet ceremony the signer's JAdES is mandatory because it carries the nonce binding; on the authenticated submission path it is optional and validated if present.

Independently of this, the **instance** signs every contract sent to peers with its HSM-backed DID key as JAdES (Chapter 08). This instance signature is a system action, not a person's AES (ADR-17). An electronic **seal** of an organization is decided as a direction but not implemented (ADR-21).

6.4 Validation: internal checks plus EU DSS

Submitted signatures pass a multi-stage acceptance check (ADR-20); every stage is a hard abort criterion:

1. **Byte pinning.** The submitted PDF must **begin** byte-exactly with the pinned TBS PDF: a PAdES signature is an incremental update that only appends bytes. The JAdES payload is checked the same way against the pinned canonical payload.
2. **Content comparison.** pdf-core compares the visible page content of the submitted document with the prepared one; nothing is re-rendered.
3. **Nonce binding.** In a published wallet ceremony, the JAdES must carry the ceremony's fresh request nonce in its signature-protected header. Someone who only knows the ceremony URL cannot forge an acceptable response without the signer's key.

4. **Externally via DSS.** The report delivers the ETSI indication, qualification, format/level, certificate subject and signing time, attributed to the most recent signature when several are present. **Without a configured DSS the submission path accepts no signature**; on the read paths, only the report is omitted without DSS.
5. **Level gate.** For **AES** the criterion is deliberately not `TOTAL-PASSED`, because that additionally requires a qualified EU trust list CA, a QES property. For AES, cryptographic integrity and unambiguous attribution suffice; an `INDETERMINATE` with a pure trust chain gap is accepted, any crypto or integrity error is rejected. For **QES**, `TOTAL-PASSED` is required **and** the qualification of a qualified certificate with QSCD.
6. **Sole-control gate.** The signing certificate is read directly from the CMS structure of the submitted PDF; its subject must match the name of the ceremony's verified identity. For QES the match is mandatory (eIDAS Annex I), for AES it is active by default and can be disabled only as a documented operator decision. The same signer must use the same certificate across all of their signatures on one contract. Subject and serial number are recorded for the compliance viewer.

A configured but unreachable DSS is an error, not a skipped check. An invalid JAdES is rejected, never silently recorded.

Subsequent verification. A signed contract can be re-verified at any time. The check reads the embedded signing summary credentials and verifies them first against their issuer's key. On top of that sit the **signer binding** (the pseudonymous identifier from the evidence against the recorded signatures), the **SHACL drift check** (repeating the validation against exactly the shapes version of the signature and comparing the findings hash; a hub version bump causes no false alarm, ADR-8) and the **DSS assessment**, if configured.

6.5 Timestamping

With `SIGNED`, the archive entry is created; its notarization evidence receives an RFC 3161 timestamp. The request runs through ORCE, and the backend verifies the response against the trusted TSA certificate. Changing the TSA therefore means two things: switching the flow in ORCE and replacing the trust anchor. The audit checkpoints use the same machinery; a temporarily unreachable TSA is caught up later (Chapter 09).

6.6 Visibility: viewers, verification, revocation

- **Signing list:** the signer's pending, completed and revoked contracts; every signature application leaves an audit entry.
- **Secure Contract Viewer:** integrity check (rebuild of the base PDF, hash comparison, C2PA and revocation check) and entry into the ceremony; the ceremony opens only after a passed check. The C2PA chain can be inspected via its own retrieval.
- **Signature Compliance Viewer:** per signature the signer, field, required and achieved level, certificate subject and serial number (sole-control evidence), the cryptographic bindings from the signing summary credential, plus integrity findings and the DSS report.
- **Revocation:** sets a signature to `REVOKED`; the envelope keeps both points in time, the action is audited. For a cross-instance contract, the revocation goes immediately to the counterparty, which adopts the state (Chapter 08).

6.7 Known limitations

- **Extraction of the embedded JSON-LD.** The comparison resolves the embedded document via the last definition of the attachment object in the file content, not via the cross-reference table. A deliberately crafted document can make this resolution diverge from that of a PDF reader.
- **More than two parties.** The evidence of the counterparty signature is held per contract, not per signature field. Contracts with three or more parties are therefore not deployed (Chapter 04).

07 Documents and Provenance

This chapter describes how the human-readable contract document (PDF/A-3) is produced from the machine-readable contract (JSON-LD), why the rendering is deterministic, how each instance independently verifies that both representations match, how origin remains traceable via C2PA manifests, and how artifacts are stored and erased. The PDF is a second representation of the same agreement, verifiable against the machine-readable form at any time, rather than a mere printout of the contract.

7.1 Components involved

Component	Role
`pdf-core`	Independent, stateless service; sole owner of all byte-level PDF work: deterministic compilation, incremental amendments, provenance re-anchor, extraction of the embedded JSON-LD, C2PA embedding, re-render and content comparison. The backend never parses PDF bytes itself
Backend / PDF generation	Orchestrates: listens to lifecycle events, has pdf-core render or amend, issues lifecycle credentials, signs the C2PA structures, stores the result encrypted, and keeps the PDF state per contract/template (CID, renderer version, C2PA state, payload hash)
Artifact store	Encrypting layer over IPFS: every artifact is encrypted before writing, every read decrypts with authentication
C2PA manifest service	Public, unauthenticated retrieval of a contract's C2PA manifest store
HSM (via the backend)	Signs the COSE structures of the C2PA manifests and unwraps the content keys. pdf-core holds no key material: it builds the structure to be signed, the backend signs
Status list (`/status-list/1`)	Signed revocation list of the instance's own lifecycle and signing summary credentials, served by the backend itself (ADR-34)

7.2 Deterministic rendering

pdf-core guarantees: **the same JSON-LD payload always produces byte-identical page content.**

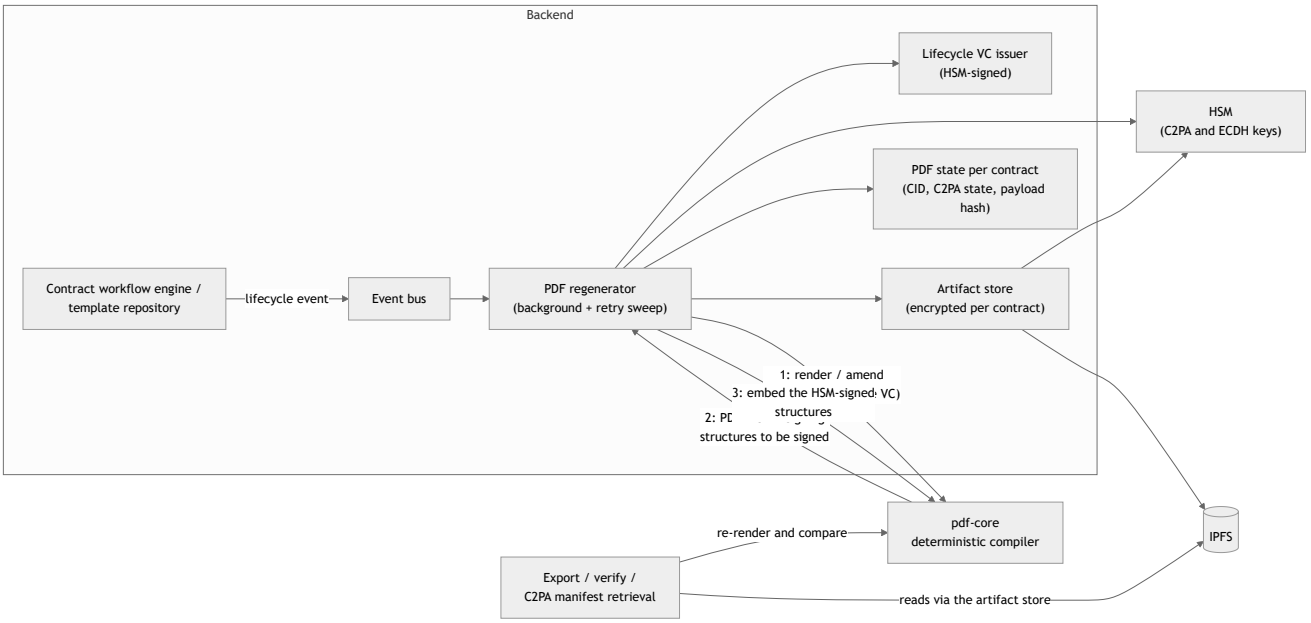
- The render time is a fixed epoch rather than the system clock; the trustworthy contract time is the timestamp of the PAdES signature (Chapter 06).
- The submitted payload is embedded **verbatim** as an attachment, exactly the transmitted bytes. Its SHA-256 is the document's content address, recomputable by any verifier from the attachment.
- Fonts are embedded (PDF/A conformance); layout and structure are derived entirely from the payload. Semantic data that is not rendered (such as ODRL policies) remains unchanged in the embedded JSON-LD.

Because the page content is a pure function of the payload, the correspondence of human- and machine-readable form can be proven at any time: extract the payload, recompile, compare page contents.

Multiple signatures are **sequential** by construction: PDF/A-3 requires every signature to cover all preceding bytes. Changes after a signature happen as incremental revisions over the signed bytes.

7.3 The rendering and provenance flow

PDFs are never created on demand in the request path; they are produced event-driven in the background. Every lifecycle event travels via the event bus into the regenerator: it issues a lifecycle Verifiable Credential, has pdf-core render or amend (the C2PA chain grows, it never starts over), signs the returned C2PA structures with the HSM and has them embedded, stores the result encrypted and updates the PDF state.



The export always serves from this store: it delivers the current PDF, waits for a running regeneration and never renders itself. A PDF already signed with PAdES is **frozen**: it is delivered unchanged even as the lifecycle state moves on. The only revision after signing is the provenance re-anchor of signature finalization (ADR-26, Chapter 06); after that nothing mutates.

7.4 C2PA manifests

Every PDF version carries a C2PA manifest store (JUMBF) covering the entire visible page content. The manifests **stack** over the lifecycle: every state change appends a lifecycle assertion together with a Verifiable Credential of the instance attesting the change, the asset hash and the time. The COSE signature of each manifest is created by the backend with the HSM-held C2PA key; the instance's certificate chain travels with every signing request to pdf-core instead of being configured into the renderer.

A signed contract carries one manifest more than an unsigned one: the lifecycle manifest the signature commits to, and the re-anchor manifest that commits to the signature (ADR-26).

Revocation via the instance's own status list (ADR-34). Every lifecycle and signing summary credential receives, at issuance, an entry in the status list the instance itself serves under `/status-list/1`; without an entry no credential is issued. The list is signed with the same HSM key and certificate chain as the

manifests; the leaf names the instance's origin and did:web, because a verifier rejects a list whose certificate does not carry the stated issuer. Terminal contract states set the revocation bit (Chapter 10). Verification reads the status from the list named by the respective credential, only after the list's signature has been checked; the anchor of the instance's own chain automatically counts among the trust anchors for this.

Provenance is delivered twice (ADR-4): embedded as JUMBF in the PDF and via the public C2PA endpoint, because an external verifier must be able to resolve the provenance chain without an account. The visibility of a chain is therefore tied to the availability of its content key (7.6).

7.5 Independent validation: human- vs. machine-readable

The consistency of both representations is enforced at three points:

1. **On demand** via the verify endpoints: extract the embedded JSON-LD, re-render, compare the page content against the stored state; additionally check the C2PA signature, the lifecycle credential and its live revocation status.
2. **On receipt of a peer PDF** (Chapter 08): the page content must be exactly the re-render of the instance's own embedded payload. The comparison considers only the page contents, so legitimate C2PA, signature and amendment layers of the sender do not trigger it.
3. **At signature acceptance** (Chapter 06): submitted against prepared document.

For a PDF with appended revisions, verification replays each revision deterministically: a revision with a new lifecycle credential as an amendment, a revision with unchanged payload as a provenance re-anchor. A signed PDF counts as good only if the only thing appended after the signature is byte-exactly the provenance this instance itself produced.

The verification result reports only what was checked. If the PDF carries no PAdES signature, or this path does not verify it cryptographically, the result reports "not available" for the PDF signature instead of an apparently passed check. It additionally names its error class directly (ADR-30):

Class	Meaning
`content_hash_mismatch`	Manifest present, but the content differs from the embedded JSON-LD
`artifact_not_authentic`	The stored bytes failed authenticated decryption; they are not the ones this instance wrote
`verification_failed`	Another check failed
(empty)	The check passed

7.6 Storage: encryption, tamper evidence, erasure

Every artifact is encrypted **before** writing (ADR-28). The content key is drawn at random per **erasure scope**: per contract, per template, and instance-wide for checkpoints and instance-level reports. The scope identifier enters the ciphertext as authenticated data, so an artifact cannot be smuggled into another scope unnoticed.

At rest, the content key exists **only wrapped** against the key agreement key published in the DID document; unwrapping requires the HSM of exactly this instance. With every contract dispatch, a copy wrapped for the peer travels along; the receiver adopts it once and re-wraps it against its own key. Both instances then hold the same content key, each copy openable only by its own HSM.

Three consequences:

- **Tampering with storage is a verification verdict, not a server error (ADR-30).** If authenticated decryption fails, the stored bytes are not the written ones; that is exactly what verification reports. Plaintext is never derived from unauthenticated ciphertext.
- **The CIDs of two instances differ** for the same contract, because each side encrypts under its own random value. This has no consequences: no CID ever crosses the peer interface.
- **Erasure is key destruction.** Erasure destroys the wrapped copies and keeps the row as a destruction record (Chapters 04 and 09). Export, verification and bundle retrieval afterwards return a defined "content erased" response; list and metadata views keep working.

Known limitation. Key destruction does not remove the ciphertexts from the IPFS node. As long as a database backup from before the destruction exists and the key agreement key lives on in the HSM, a readable key could be reconstructed from it. The erasure promise reaches as far as the retention window of the backups; catching up erasure markers after a restore is described in the backup guide.

7.7 Interfaces and artifacts

Backend (authenticated, role-based; see Appendix A):

Endpoint	Purpose
`GET /pdf/export/contract/{did}`, `GET /pdf/export/template/{did}`	Current PDF (PDF/A-3, embedded JSON-LD, C2PA chain)
`GET /pdf/verify/contract/{did}`, `GET /pdf/verify/template/{did}`	Text/data and provenance verification including the error class
`GET /contract/export/{did}`	ZIP bundle: JSON-LD, signed PDF, C2PA store, credentials, signature states, manifest with a hash per entry, hierarchy relatives
`GET /template/export/{did}`	ZIP bundle of a template
`GET /c2pa/manifest/{contract_did}`	Public C2PA manifest store; optionally the chain listing

pdf-core (internal, called exclusively by the backend), by task group:

Group	Endpoints	Purpose
Render	`POST /render`, `/render/amendment`, `/render/reanchor`	Compile, extend incrementally, provenance re-anchor (ADR-26)
Verify	`POST /verify`, `/verify/content`, `/verify/content-match`	Re-render verification; receipt check for peer PDFs; comparison of two PDFs at signature acceptance
Extract and bind	`POST /payload/extract`, `/manifest/extract`, `/manifest/chain`, `/claim`	Read out embedded JSON-LD, C2PA store and chain listing; bind an external payload to a PDF
Embed	`POST /c2pa/embed`, `/evidence/embed`, `/evidence/extract`	Insert COSE structures signed by the backend; embed and read out signature evidence
Meta	`GET /version`, `GET /ontology/…`	Renderer version (part of the persisted PDF state) and schema artifacts of the renderer

The connection settings of pdf-core are deployment configuration (deployment guide).

7.8 Operational behaviour

- **Export waits:** while a regeneration is running, the export polls and aborts after its time window with a clear error; the blocking condition is logged.
- **Verify requires an existing store:** without a prior export, verification fails with a corresponding message. If the lifecycle state has advanced, it regenerates transparently.
- **A sweep catches up lost regenerations:** a periodic pass finds entities whose stored PDF does not match the current document, processes them in bounded batches and retries each one a limited number of times with growing intervals. A permanently unrenderable contract does not block the healable ones.
- **IPFS is eventually consistent:** freshly written CIDs are not always immediately readable; the client retries with backoff.
- **Bundle exports refuse rather than deliver incompletely:** if a referenced component is missing, the export responds with a findings list instead of a gappy archive.
- **An amendment without a content change** is rejected; the deliberate exception is the provenance re-anchor with its own endpoint.
- **A faulty peer PDF is a reason for rejection, not a crash:** the processing of incoming documents is bounded in size and effort.

08 Federation

This chapter describes how two independent DCS instances exchange and negotiate contracts, which trust model secures every interaction, and how templates become discoverable via the XFSC Federated Catalogue. The basic principle: **artifacts cross the instance boundary, state does not.** Each instance runs its own workflow, its own RBAC and its own copy of the contract (ADR-13).

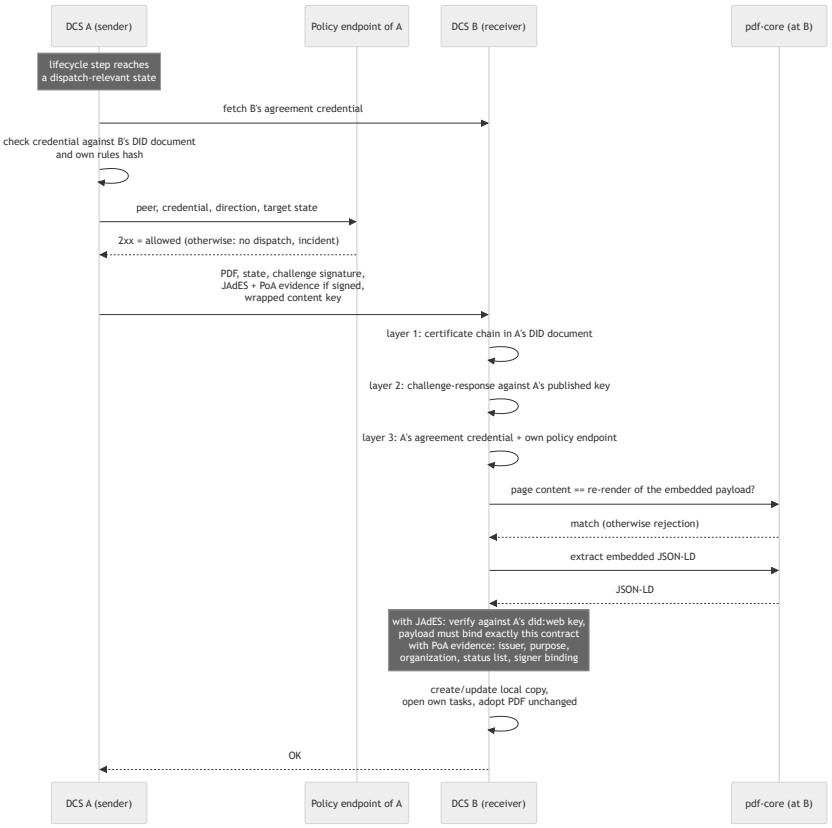
8.1 Components involved

Component	Role
DCS-to-DCS synchronizer	Outbound side: listens to lifecycle events and dispatches the contract PDF; failed attempts land in a retry table
Peer endpoints	Inbound side: authenticate the sender, verify the PDF (and, for signed dispatches, JAdES and PoA evidence), adopt it as a local copy; accept erasure requests
Trust gate	Trust layer in front of every interaction, in both directions: the peer's agreement credential and the local policy endpoint (fail-closed)
Counterparty PoA gate	Verifies the power of attorney behind every signature a counterparty sends along (ADR-31)
did:web identity + HSM	Instance identity with certificate chain; private keys in the HSM (Chapter 05)
pdf-core	On receipt, checks that the page content is the re-render of the embedded payload, and extracts the JSON-LD (Chapter 07)
Audit trail	Every trust gate rejection is recorded as an incident (Chapter 09)
Federated Catalogue	Cross-instance template discovery

8.2 Instance-to-instance exchange: the PDF is the wire format

What is exchanged: the **contract PDF per visible lifecycle step**, the **JAdES signature after signing**, and the **power-of-attorney evidence** behind that signature. The PDF is self-contained: it holds the JSON-LD, the C2PA chain and any signatures. A bare PDF is a proposal; a PDF with JAdES is the counterparty's signature. In addition, the content key wrapped for the peer travels along (ADR-28, Chapter 07).

Only what the other side must see is dispatched: the states `OFFERED`, `NEGOTIATION`, `SIGNED` and `REVOKED`. Internal states (draft, review, approval, activation, termination) stay local. Negotiation proceeds in three phases: negotiate (counter-proposals as new PDF versions), agree (both sides confirm the same version), and only then sign; a unilaterally signed, not yet agreed contract would be a disputable artifact.



On **first receipt**, the local copy is created in state `OFFERED` with the sender as origin; the receiver's own review, approval and negotiation tasks are opened locally. A **repeated receipt** updates the content and increments the local version without overwriting the receiver's own workflow progress. The state declared by the sender is purely informative, with exactly one exception: if the authenticated counterparty reports `REVOKED`, the receiver adopts this state, because the revocation invalidates the agreement.

The received PDF is stored as the local copy **exactly as it arrived**. Re-rendering would destroy the counterparty's C2PA chain; later local changes amend this base, so the chain grows across both instances.

Transport follows identity. The peer is resolved from its `did:web` identifier, including path segments, so several instances can share one host. Resolution is HTTPS-only; the only exceptions are loopback hosts and hosts a deployment explicitly names as insecure (cluster-internal identities). Outbound fetches follow no redirects, select no addresses where a published identity never resides, and are bounded by timeouts. Authentication is not at transport level; it uses a challenge-response signature in the request itself.

8.3 Trust model between instances

Trust arises exclusively from verifiable layers:

Layer	Question	Mechanism	Anchor
1: Identity	Who are you?	`did:web` with certificate chain, optionally checked against the EU trust pool	EU trust lists / QTSP
2: Proof of possession	Is it you speaking right now?	Challenge-response per request: random value signed with the sender's HSM key, verified against its published key	Layer 1
3a: Rule acceptance	Do you play by the rules?	Self-signed agreement credential under `/.well-known/dcs-agreement-credential.json` whose `termsOfUse` hash names the federation rules	Rules compiled into the binary; the hash must match the instance's own
3b: Policy	May **this** interaction take place?	The instance's own local policy endpoint (`DCS_TRUST_PDP_URL`): 2xx allows, anything else denies	Whatever the endpoint consults (allowlist, trust lists, clearing house, …)
3c: Power of attorney	Was the signer allowed to act for this party?	PoA evidence sent along, verified against an issuer admitted for `peer` with authority for the organization, including the status list (ADR-31)	Issuer trust of the receiving instance
4: Legal effect	Does the contract bind?	AES/QES on the document (PAdES/JAdES, Chapter 06)	eIDAS

Layers 3a/3b form the **trust gate** (ADR-19), consulted before every dispatch and on every receipt:

- **Fail-closed.** An unconfigured, unreachable or unresponsive policy endpoint denies like an explicit deny. Without a configured policy endpoint, federation is effectively switched off.
- **Rule acceptance is bound to the software version.** The rules are compiled in and retrievable under `/.well-known/dcs-federation-rules.md` ; rule changes are software releases.
- **The credential must come from the peer itself and be time-limited.** Issuer and retrieval source must point to the same resolution target, the proof must reference an assertion key of the peer's DID document, and a `validUntil` is mandatory: expired, unlimited or limited longer than the federation allows means rejected. The instance reissues its own credential before expiry.
- **No "phone home".** Every instance consults only its own policy endpoint. The default delivery contains a minimal flow that operators extend.
- **Every rejection is auditable.** On the dispatch path, an agreement credential rejection is retried; a policy endpoint rejection is terminal: no retry, exactly one incident per attempt, deduplicated per contract, peer and direction.

Provenance and power of attorney of the counterparty artifact. The JAdES of a signed dispatch is fully verified before acceptance: against the instance's own certificate chain, with the sender's published `did:web` assertion key as the leaf, and the signed payload must bind exactly the contract embedded in the PDF. The challenge-response authenticates only the session; the JAdES binds the contract **content** to the sender's key and remains retrievable as independently verifiable evidence.

The PoA evidence sent along (ADR-31) is verified before anything is persisted: issuer admitted for `peer` with authority for this organization, valid signature and validity period, not revoked, authorizes exactly the party that signed, and held by exactly the stated signer. What the check establishes is an attestation of authority; who the person is was established by the instance where the ceremony ran. The failure behaviour is deliberately asymmetric: **present but unverifiable** evidence denies the dispatch and creates

an incident; **missing** evidence does not, because a peer without retained evidence must be able to keep federating, and a party without authority is flagged by the compliance viewer. If a PoA credential expires or is revoked after signing, later dispatches of the same contract fail; the lifetime of a power of attorney must outlast the exchange.

8.4 Erasure across the instance boundary

The erasure path is its own peer call with the same did:web challenge-response as the PDF exchange:

1. The requesting instance destroys its own wrapped content keys and writes a destruction event into the audit trail. If this local step fails, it is a hard error.
2. It requests every counterparty instance to do the same. An unreachable counterparty does not block: the request stays open and is redelivered periodically.
3. Both sides emit the same destruction event, with actor, contract, scope and reason, never with content.

A destroyed scope is final: neither a local write nor a key sent along by the peer creates a new one. The progress of the chain is visible via a status query (Chapter 09); Chapter 07 describes the limits of the promise.

8.5 Federated Catalogue integration

The Federated Catalogue serves template discovery, not business data storage: it verifies and stores self-descriptions and makes them queryable; several FC instances can synchronize with each other. The data model links the template (Resource) with the Participant and a ServiceOffering carrying the endpoint URL of the operating DCS instance. From a found template one can thus navigate to the responsible DCS; the usual adoption is registration as an own template into the own repository with an own approval process (Chapter 03).

For authentication the instance uses a Keycloak service account holding all functional catalogue roles including the administrative ones (ADR-18). A hard operational limit follows: a catalogue must not be shared by several DCS deployments that do not fully trust each other. The deployment enforces this: a catalogue integration against a remote catalogue that is not part of the same deployment fails as long as the operator has not explicitly confirmed the shared administrative trust boundary.

Operational details of the catalogue coupling are collected in the FC integration guide and the deployment guide.

8.6 Operational behaviour

- **Dispatch is event-driven and idempotently recoverable.** A failed dispatch leaves a retry entry that a scheduler retries periodically; success clears it. This delivery is database-backed and survives lost event bus messages.
- **An offer before the PDF is ready is never silently dropped:** the dispatch is explicitly queued for retry until the PDF exists.

- **Terminal policy denials leave the retry queue** atomically together with the incident; otherwise they would run again forever.
- **What the operator sees:** trust gate incidents in the audit trail with peer DID, direction and reason; dispatch and receipt errors in the logs; on a content mismatch a diagnosis with the diverging location; open erasure requests in the erasure status query.

09 Audit and Compliance

The DCS treats traceability as a cryptographic property: every business-relevant event is anchored in a tamper-evident audit trail whose integrity is independently verifiable via hash chaining, Merkle checkpoints and trusted timestamps. This chapter describes the audit trail, audits and reports, continuous compliance monitoring and policy enforcement via ODRL and OPA.

9.1 Components involved

Component	Role
Business components	Write their events transactionally into an outbox table, in the same database commit as the business state change
Outbox processor	Publishes events on the event bus and anchors audit-visible events as audit entries
Artifact store over IPFS	Holds audit entries and checkpoints content-addressed; the CIDs are the authority of the trail
PostgreSQL	Index over the trail: chain heads, checkpoints, pending timestamps, dead-letter status, risk register, workflow gate runs
TSA (RFC 3161, via an ORCE flow)	Timestamp over every checkpoint root
ORCE, external notary	Publishes the checkpoints sequentially to an external sink outside the operator's reach
ORCE, audit executor	Assesses an audit retrieval: the DCS collects the evidence, the executor forms the findings
ORCE, workflow gate executor	Decides on the contract snapshot before every gated transition (Chapter 04)
OPA (embedded)	Evaluates ODRL constraints as Rego policies
`pacmonitor` (CronJob)	Runs the compliance sweep on schedule (ADR-32)
Process Audit & Compliance (API)	Audits, reports, monitoring, incident reports, checkpoint head and inclusion proofs, workflow gate runs

9.2 The audit trail

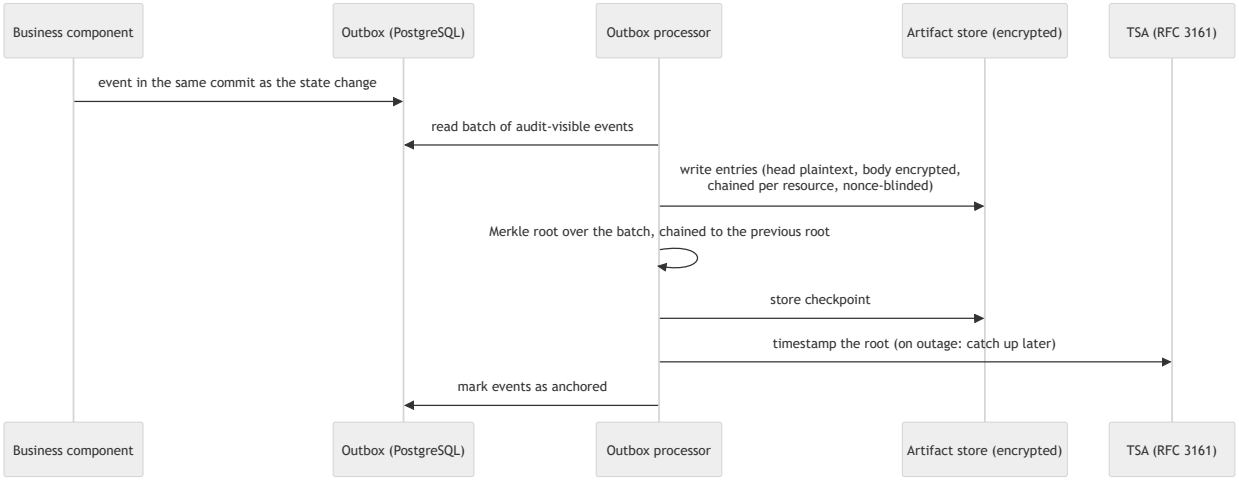
Every business operation writes its event in the same transaction as the state change (outbox pattern); an event cannot get lost without the state change being missing too. The outbox processor works in three decoupled loops: **publish** (events onto the event bus, high frequency, never waits for TSA or storage round trips), **anchor** (audit-visible events in batches as audit entries plus a Merkle checkpoint; pure lookup events are not anchored) and **re-timestamp** (checkpoints anchored without a timestamp during a TSA outage).

Hash chaining per resource. Every audit entry references its predecessor of the same resource by CID. A retroactively changed entry would have a different CID, and all successors would point into the void. Chains of different resources are independent and written concurrently; an audit read walks the chain

backwards from the head.

Public head, private body (ADR-28). The head of an entry (component, event type, DID, timestamp, predecessor CID, blinding nonce) is plaintext; the body with the event payload is under the content key of the respective contract or template. Events without a resource reference, including the erasure records themselves, are under the instance-wide key, which no erasure request destroys. Leaf hashes and checkpoints are computed over the stored object as it is: key destruction makes the bodies unreadable while every inclusion proof continues to verify. Tamper evidence and erasability coexist.

Merkle checkpoints and timestamps (ADR-16). Per anchoring batch a Merkle root is computed (with domain separation between leaf and node hashing), chained to the root of the previous checkpoint: a single published root transitively commits the entire log before it and proves its append-only property. The root is timestamped once per batch; during a TSA outage the checkpoint is anchored anyway and the timestamp supplied later. Every entry carries a random **blinding nonce** in its leaf hash, because audit entries are highly guessable; with the nonce, inclusion proofs can be published safely.



What can be published:

Publishable	Never published
Sequence number, root, predecessor root, leaf count, creation time, RFC 3161 token	Leaf CIDs (retrieval capabilities into the instance's own store)
Inclusion proofs (hashes only)	The entries themselves (contain DIDs, identities, workflow payloads)

The proof retrieval delivers leaf hash, leaf index, sibling hashes and the corresponding head, never the entry, and is verified internally against the stored root before release.

External anchoring. A trail held only by the operator proves nothing against the operator. An ORCE flow therefore periodically reads the checkpoints via the compliance API (as a machine identity with the role **Sys. Auditor**, which reaches only the checkpoint view) and publishes the public projection of every checkpoint in strict sequence to a configured, authenticated HTTPS sink: gapless, with an idempotency key and a persisted confirmation state, so a lost response or a restart appends no checkpoint twice. If the sink reports a sequence gap or a diverging predecessor root, publication stops visibly instead of extending a

broken external chain. A third party verifies an entry with the entry bytes including the nonce, the inclusion proof and a checkpoint obtained from the external sink, not from the operator. Without a configured sink, no external anchoring takes place.

9.3 Audits and reports

Audit retrieval. Auditors request the audit trails of a scope (templates, contracts, archive, signatures; optionally filtered to a DID). Every request requires a justification, and the retrieval itself is written into the trail as an audit event. The retrieval is two-stage: the DCS collects the evidence (hash chains plus freshly computed content and policy checks) and hands it to the external **audit executor**, which forms the findings; request, response and raw response are persisted. If no executor is configured or reachable, the retrieval answers with its own error; there is no silent fallback to an internal assessment.

Reports. The report path produces reports in JSON, CSV or PDF with a deterministic report ID and content hash; the generation is recorded in the trail with hash, CID and justification. A handed-out report is thus verifiable against the trail later.

Workflow gate runs as evidence. Every gate run (Chapter 04) is itself a compliance artifact with snapshot identity, local assessment and executor response. A compliance officer can decide a run in state `REVIEW` with a justification; the decision resumes exactly the halted transition.

9.4 Continuous monitoring and incidents

The compliance sweep reports four risk classes:

- `MISSING_APPROVAL`: contracts waiting for a pending required approval (one risk per contract-approver pair).
- `CONTRACT_UNDERPERFORMANCE`: active contracts for which the target system has reported a rule violation via the deployment callback (9.5). The violation is reported as an alert, not rejected as a request error, because the contract is in force and the violation is a fact to be documented.
- `CONTRACT_DEPLOYMENT_FAILED`: deployments whose dispatch to the designated target system failed.
- `UNAUTHORIZED_ACCESS`: contracts with persisted access denials from the party check (Chapter 04), one risk per contract-actor pair.

Scheduled sweep rather than on-demand only (ADR-32). The same detection logic additionally runs on schedule: a dedicated command (`pacmonitor`) performs exactly one sweep and exits. It opens only the database and does not run the server's startup checks, so monitoring keeps working exactly when a neighbouring component is disrupted; detections go into the outbox, and the running server anchors and distributes them. A **CronJob** invokes the command at a fixed interval and is active in the delivered state; switching it off means risks are found only on demand.

A **risk register** keeps one row per open violation, keyed by contract, risk class and a hash of the detail text (one contract can carry several risks of the same class). A risk alerts on first detection and again when it recurs after remediation, never on the sweeps in between. A detected risk is **not a job failure**: the sweep ends successfully; a failure means the sweep could not run. Every sweep is itself anchored as an event,

every risk additionally in the audit chain of the affected contract. The response of the on-demand retrieval remains complete; deduplication only controls alerting. Where second-level latency is required, the way is an event bus subscriber alongside the sweep, not a tighter interval.

Alerts to external subscribers. The webhook platform delivers, alongside the lifecycle events, `compliance.risk_detected` on first detection of a risk; deliveries are logged and can be acknowledged (Appendix A).

Incident reports. Findings outside automated monitoring are filed by the compliance officer as an incident report: findings with a risk class and description, linked to the DID of the affected resource. Every finding is anchored as its own event in that resource's audit chain.

Startup attestation of the configuration. At startup the backend hashes the security-critical mounted configuration artifacts (DID document, issuer trust document, certificate trust anchors, authorization policy) and anchors the hashes as an attestation event in the trail. If the operator pins expected hashes, a missing or diverging pinned file aborts startup, as does a syntactically invalid pin list. The policy is attested as well because it ranks above the trust document (Chapter 05).

Erasure records. The destruction of a contract's content keys produces an event on every participating instance with actor, contract, scope and reason, never with content, stored under the instance-wide key. The progress of the erasure chain across the instance boundary is visible via a dedicated status query.

9.5 Policy enforcement: ODRL on OPA

Contract terms are ODRL 2.0 under a DCS profile (ADR-6), evaluated by Open Policy Agent embedded as a library (ADR-11): no sidecar, no additional latency on the approval and signing path. The Rego module answers the truth of **one** constraint; the deontic logic above it (prohibitions, `and` / `or` / `xone`, duties, usage-time operands) resides in the evaluating code.

Enforcement is split between the DCS and the target system (ADR-33):

Moment	Who decides	Effect
Before signing (approval, signing preparation)	The DCS evaluates the field values carried inline in the contract against its own terms	Server-side rejection of constraint-violating values; findings in the trail
Execution / KPI monitoring	The target system enforcing the rules; the DCS records its verdict	Reported verdicts feed the risk register and the contract's audit chain

The target system classifies, the DCS records (ADR-33). Constraints whose inputs arise only at usage time (time, place, quantity, delivered performance) can only be decided by the enforcing party: the DCS holds the terms, the target system holds the events. A KPI report therefore carries, alongside measured value and time, the target system's verdict (`satisfied`, `violated`, `not_evaluated`) and the `@id` of the ODRL rule it refers to, verbatim from the policy of the deployment envelope. The DCS checks attributability and records rather than recomputes: a verdict for a rule this contract never deployed is rejected as a request error; a report without a verdict is recorded as `not_evaluated`, never as fulfilment. Before signing, such constraints appear as a deferred finding, not as a passed check; a target system that reports nothing leaves the contract unobserved, and that is what the DCS states.

In addition, two versioned **policy sets** under `docs/policies/` define the instance-wide check rules (template and contract content set). Every rule carries a stable rule ID and a severity; every finding is recorded as an audit event, and a finding of severity "error" blocks the corresponding workflow gate transition (Chapter 04). These rules check **structure and presence** of declared fields, not their values; value limits are enforced where they are expressed as SHACL (Chapter 03).

9.6 Interfaces

Endpoint	Purpose	Roles
`POST /pac/audit`	Retrieve the audit trail of a scope and have it externally assessed (justification mandatory, retrieval is audited)	Auditor, Archive Manager
`GET /pac/report`	Generate an audit report (JSON/CSV/PDF; hash and CID in the trail)	Auditor, Archive Manager
`POST /pac/report`	Incident report: record non-compliance findings for a resource	Compliance Officer
`GET /pac/monitor`	Compliance sweep on demand; a run without findings is audited as well	Compliance Officer
`GET /pac/audit/checkpoint/head`	Latest checkpoint head (hashes, counts, timestamps only)	Auditor, Archive Manager, Sys. Auditor
`GET /pac/audit/checkpoint/{seq}`	A specific checkpoint	Auditor, Archive Manager, Sys. Auditor
`GET /pac/audit/checkpoint/proof/{entry_cid}`	Merkle inclusion proof for an anchored entry	Auditor
`GET /pac/workflow-gates/{run_id}`	Read a persisted workflow gate run	Compliance Officer
`POST /pac/workflow-gates/review`	Record a decision on a run in state `REVIEW`	Compliance Officer
`GET /archive/erasure-status`	State of a contract's key destruction, locally and per peer	Archive Manager, Auditor

9.7 Operational behaviour

- **Transient anchoring errors** are counted per event and retried on the next tick; the entry moves into the next checkpoint. Its business timestamp remains unchanged.
- **Dead-lettering:** after 50 failed anchoring attempts an event is dead-lettered and logged prominently. Dead-letter events are gaps in the trail and need an operator; they can be found in the outbox table by their marker together with the last error.
- **TSA outage:** checkpoints are anchored without a timestamp and re-stamped later; the state is visible in the head.

- **Volume leak:** published heads reveal batch size and cadence, hence activity volume. This is a deliberate trade-off (ADR-16).
- **Read limits:** a full-trail read walks at most 500 checkpoints backwards; component-wide audits read the resource chains in parallel.
- **Erased contracts in the audit:** the chain remains present and provable, its bodies are unreadable; the audit chain continues to document the erasure.

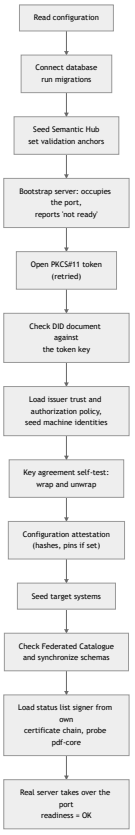
10 Operations and Monitoring

This chapter describes how a DCS instance behaves in operation: what happens at startup, which background processes run, which signals exist and how an operator recognizes a failure.

It deliberately contains no commands, values or procedures. Installation, configuration reference, environment matrix, upgrade and rollback are in the deployment guide; backup and restore in the backup guide.

10.1 Startup behaviour: fail fast, patient in exactly one place

The backend checks its hard dependencies itself and never keeps running degraded:



The background processes (10.3) start step by step during this initialization, not at a single point; the instance reports ready only after the last stage. Key properties:

- **The port is occupied from the start.** A bootstrap server answers the readiness endpoint with "not ready" while initialization runs. Kubernetes can thus distinguish a starting process from an operational DCS.
- **Exactly one stage waits instead of aborting:** opening the PKCS#11 token, because on a fresh installation the token provisioning may still be running. Everything after that is hard.
- **If the DID document does not match the token key, the instance does not start.** This is the most common consequence of re-provisioning the token without regenerating the document.
- **The key agreement self-test is decisive:** if it fails, no stored artifact would be readable, and the instance does not start (Chapter 07).

- **A configured Federated Catalogue is a hard dependency:** incompletely configured or not functionally reachable means no start; not configured at all means operation without catalogue functions.
- **pdf-core is probed via HTTP**, with retries over a bounded window; if it stays unreachable, the process exits with a clear message. An unreadable status list certificate chain also means no start (Chapter 07).
- **The audit and workflow gate executors are mandatory:** if their address is missing, the instance does not start. They would otherwise sit as fail-closed gates in every lifecycle transition (Chapters 04 and 09).
- **Invalid declarations abort startup** instead of taking effect silently: an unknown role in the machine identity seed, a faulty target system declaration, an authorization policy that does not compile, a syntactically wrong pin list, a trust entry with an unresolvable mechanism.

10.2 Probes and readiness

Signal	Meaning
Readiness endpoint answers "not ready"	The process is running, initialization is not complete. Normal during installation and restart; if persistent, it is a symptom (10.6)
Readiness endpoint answers OK	All startup checks passed, the real server serves the API
Process exits with an error message	A hard startup dependency is missing or wrong. The message names it; the orchestrator restarts the container

The backend container is checked for readiness only; no liveness signal is probed. A process that is running but no longer working is therefore never restarted automatically; if it still answers on the readiness endpoint, it even accepts requests. This follows from the bootstrap behaviour: a liveness probe against the same endpoint would kill a process that is correctly waiting for a not-yet-provisioned dependency. An operator who wants a liveness probe adds it deliberately and must cover the startup case.

pdf-core has both readiness and liveness probes; the bundled companion services bring their own probes.

10.3 What runs permanently in the background

Knowing these loops explains almost every behaviour that looks "delayed" from outside.

Process	Task	Behaviour under disruption
Outbox publication	Put events on the event bus	High frequency; an unreachable bus delays subscribers, not the business logic
Outbox anchoring	Write audit entries, build Merkle checkpoints, timestamp roots	Counts failures per event; after 50 attempts dead-letter with a prominent log
Timestamp catch-up	Stamp checkpoints anchored without a timestamp	Runs until the TSA answers again
PDF regeneration	Render/amend on lifecycle events	An event is delivered at most once; failures are caught by the retry sweep
Regeneration retry sweep	Find entities whose stored PDF does not match the current document	Bounded batches, bounded attempts, growing interval
Federation dispatch	Deliver contract PDFs to peers	Failures land in the retry table; a scheduler retries them
Erasure retry	Redeliver open peer erasure requests	Own queue, same cadence as the federation retry
Status list publication	Set revocation bits of self-issued credentials in the instance's own status list	Database-backed queue; failures remain and are retried with backoff
Expiration job	Set contracts to `EXPIRED` after their expiration date	Logs errors per contract and continues; without an expiration policy the contract is skipped (Chapter 04)
Webhook delivery	Deliver events to registered subscribers	Deliveries are logged and can be acknowledged
Compliance sweep (CronJob)	Detect compliance risks on schedule (ADR-32, Chapter 09)	Own command, own pod; a detected risk is **not** a job failure

10.4 Metrics

The backend exposes a Prometheus endpoint with two basic HTTP metrics: request count and duration by method, path and status code. This makes error rates, latency distributions and load profiles per endpoint analysable (share of 429 from the request budget, share of 5xx, latency of the export and signing paths). The endpoint lies outside the authenticated API and the request budget; scraping is set up by the operator in their own monitoring infrastructure.

Application-level metrics (anchored checkpoints, retry queue lengths, open risks) are **not** exported as time series; these observations run via logs, the compliance API and the database (10.5).

10.5 What the operator sees

Logs that matter:

- every anchored checkpoint with sequence, entry count, root and timestamp status; if these messages stop while the system produces events, anchoring is stalled

- every dead-lettered audit event, unmistakably worded; it is the only way the trail can acquire a gap and needs an operator
- wait messages of token opening at startup
- retries of the federation dispatch and the regeneration sweep; caught-up TSA timestamps
- the diagnosis for a content mismatch of a received peer PDF

States in the database:

Area	Question it answers
Outbox with dead-letter marker	Are there events that never reached the trail, and why?
Retry tables for federation dispatch and peer erasure	Which dispatches or erasure requests are stuck, since when and with what error?
Risk register of compliance monitoring	Which violations are open, since when, which were closed?

API views: checkpoint head (a missing timestamp shows a TSA disruption), compliance sweep on demand, workflow gate runs (why a transition was blocked or is waiting), erasure status, key inventory, webhook delivery log.

Events for external systems: registered subscribers receive named lifecycle events and the compliance alert on first detection of a risk (Appendix A). **Subscriptions and the delivery log live in process memory**: they do not survive a restart and are not shared between replicas. Anyone who needs durable delivery re-registers after every restart.

10.6 Failure patterns

The following patterns are derived from the behaviour of the artifacts or were observed while operating the demonstration environment. Concrete remediation steps are in the deployment guide.

Symptom	Cause	Direction of remediation
Pod runs, readiness stays negative permanently, log repeats wait messages about the PKCS#11 token	The token is not provisioned or not mounted	Check the provisioning job and the token volume
Process exits immediately with a message about the DID document	The DID document does not match the key in the token, typical after re-provisioning without regenerating the document	Re-derive the DID document from the token
Process exits with a message about key agreement	Published `keyAgreement` key and token key do not match, or the token cannot perform the derivation	Reconcile document and token; on a production HSM check the derivation capability for the curve
Process exits with a message about the Federated Catalogue	Catalogue incompletely configured or not functionally reachable	Complete the catalogue configuration or disable the catalogue
Process exits with a message about a pinned configuration file	An attested file is missing or diverges from its pin; or the pin list is syntactically wrong	Fix the file or the pin (Chapter 09)
Every lifecycle transition is blocked although the contract is correct	The workflow gate executor is unreachable; the gate is fail-closed	Check the executor flow; the gate run names the cause
An audit retrieval answers with an executor error	The audit executor is unreachable or does not answer usably	Check the executor flow
A contract cannot be exported and is not dispatched to the peer	PDF regeneration failed	The retry sweep catches up; the cause is in the regenerator log
Export answers "currently regenerating"	A regeneration ran longer than the export's wait window	Retry; if persistent, it points to a pdf-core or storage disruption
Verification reports "artifact not authentic"	The stored bytes are not the written ones: manipulation or substitution in storage	Treat as a security incident; the Merkle proofs remain valid (Chapters 07, 09)
Export, verification or bundle answer "content erased"	The contract's content keys were destroyed	Expected behaviour after an erasure; the erasure status shows time and actor
Peer dispatch is stuck in the retry table	Peer unreachable, DID resolution fails or the agreement credential is rejected	Check peer reachability and DID resolution; details in the log and the retry entry
Federation is switched off although configured	The trust PDP is not set or unreachable; the gate is fail-closed	Check the policy endpoint; every rejection is an incident in the trail
A counterparty dispatch is rejected although the peer	The PoA evidence sent along does not verify: issuer not admitted for `peer`, organization not granted, credential	Check the receiving instance's issuer trust

Symptom	Cause	Direction of remediation
is known	expired or revoked	(Chapters 05, 08)
A wallet login fails reproducibly	Issuer not admitted for `login`, organization not granted, status list unreachable, or a signature (credential or status list) does not verify against the trust anchors: the anchors must cover each issuer individually, and same-named roots of different issuers can only be told apart by fingerprint	Check the trust configuration; the error names the stage
Responses with `429`	The per-credential request budget is exhausted	Check the calling behaviour; the budget is configurable
Webhook subscribers receive nothing after a restart	Subscriptions live in process memory	Re-register
Signature submission is rejected although the document is validly signed	No DSS configured or unreachable; or the byte prefix does not match because a different document than the prepared one was signed	Check DSS; the signer must sign exactly the prepared document (Chapter 06)
External PDF checkers report "changed after signing"	The provenance re-anchor after the signature (ADR-26)	Expected behaviour; the instance's own verification proves exactly this one revision (Chapter 07)

10.7 Capacity and operational limits to know

- **One backend replica per instance.** The webhook subscriptions live in process memory; multiple replicas would have different views. Scaling is vertical.
- **Anchoring is strictly sequential** and depends on TSA and storage round trips; large backlogs work off in batches.
- **A full-trail read is bounded** (at most 500 checkpoints backwards) so an audit retrieval stays within its deadline.
- **Backups delay the completion of an erasure.** A key destroyed today still exists in yesterday's backup; after a restore, the erasures performed since then must be re-applied (backup guide).
- **Key destruction removes no ciphertexts from storage**; what is removed are the archive snapshots (Chapter 07).

Appendix A Interface Reference

This appendix lists the interfaces of a DCS instance on three levels: the backend's HTTP API groups, the events on the internal event bus and the public well-known endpoints. It describes groups and purpose, not individual parameters; the complete, machine-readable API description is served by every running instance itself (Swagger UI and OpenAPI 3 specification).

Unless noted otherwise, all business APIs are protected by the OIDC bearer token; the scopes in the access token correspond to the user's role claims. Machine callers authenticate with client credentials tokens of their own OAuth2 clients; the backend reads their roles from the machine identity registry, keyed by the client ID. Public are the well-known endpoints, the C2PA manifest retrieval, the status list, the Semantic Hub resolution and the callback paths invoked by wallets and peer instances.

A.1 HTTP API groups

Group	Base path	Purpose
Auth	`/auth/...`	End-user login: OIDC session management and OpenID4VP presentation (incl. PID presentation)
TemplateRepository	`/template/...`	Lifecycle of contract templates
SemanticHub	`/semantic/...`	Versioned JSON-LD contexts, SHACL shapes, ontologies, validation profiles, clause catalog
TemplateCatalogueIntegration	`/catalogue/template/...`	Read integration with the XFSC Federated Catalogue
ContractWorkflowEngine	`/contract/...`, `/machine-identities/...`	Contract lifecycle; target system registry and machine identities
SignatureManagement	`/signature/...`	Signing ceremonies, verification, provenance, compliance
PDFGeneration	`/pdf/...`, `/contract/export/...`, `/template/export/...`	Export and independent verification of the PDF representations
ContractStorageArchive	`/archive/...`	Long-term archive of concluded contracts, erasure and erasure status
ProcessAuditAndCompliance	`/pac/...`	Audit retrieval, reports, monitoring, Merkle checkpoints, workflow gate runs
DcsToDcs	`/peer/...`	Federation: PDF/JAdES exchange and erasure requests between DCS instances
KeyInventory	`/admin/hsm-keys`	Read-only inventory of the HSM keys (Sys. Administrator)
DIDService	`/.well-known/...`	DID document, agreement credential, federation rules (public)
C2PAService	`/c2pa/...`	Public retrieval of a contract's C2PA manifest
Webhook platform	`/orce/...`	Subscriptions, deliveries and callbacks for external event receivers

Auth (`/auth/...`)

Two flow families: the OIDC login with OpenID4VP presentation and the independent, one-off PID presentation without a session.

Endpoint group	Endpoints	Purpose
Login and OIDC session	`POST /auth/login`, `/auth/login/renew`, `/auth/login/challenge`, `GET /auth/login/status`; `GET /auth/consent`, `/auth/callback`, `POST /auth/refresh`, `GET /auth/logout`, `/auth/logout-complete`	Start and renew the OID4VP login and bind it to the Hydra challenge; status polling; consent and callback; token refresh via HttpOnly cookie; logout
Wallet and PID presentation	`GET/POST /auth/presentation/request/{state}`, `POST /auth/presentation/callback`; same mechanics sessionless under `/auth/pid/presentation/…`	Signed authorization request object (request by reference) and direct-post return channel; the PID variant without a session (Chapter 05)

TemplateRepository (`/template/...`)

Endpoint group	Endpoints	Purpose
Creation and maintenance	`POST /template/create`, `POST /template/copy`, `PUT /template/update`, `POST /template/update_manage`	Create, copy, edit a template (content or management data)
Approval process	`POST /template/submit`, `POST /template/verify`, `POST /template/approve`, `POST /template/reject`	Submit for review, semantically verify, approve or reject
Retrieval and search	`GET /template/search`, `GET /template/retrieve`, `GET /template/retrieve/{did}`, `GET /template/{did}`, `GET /template/history/{did}`	Search templates, list them, resolve by DID, view history
Provenance and audit	`GET /template/provenance/{did}`, `GET /template/audit`	Provenance record and audit entries of a template
Distribution	`POST /template/register`, `POST /template/publish`, `POST /template/archive`	Register a version, publish it to the Federated Catalogue, retire it

SemanticHub (`/semantic/...`)

All read endpoints of this group are public: created artifacts carry hub anchors that external verifiers must be able to resolve without a DCS login. Only schema management writes (Template Manager).

Endpoint group	Endpoints	Purpose
Schema management	`POST /semantic/schema/register`, `POST /semantic/schema/rollback`	Register a new schema version, roll back to an earlier one
Schema retrieval	`GET /semantic/schema/retrieve`, `GET /semantic/schema/versions`, `GET /semantic/schema/list`	Retrieve the active or a named version, version and schema list
Artifact resolution	`GET /semantic/context/{name}`, `GET /semantic/shapes/{name}`, `GET /semantic/ontology/{name}`, `GET /semantic/profile/{name}`	Context, shapes, RDF configuration and validation profile under stable anchors
Clause catalog	`GET /semantic/clauses`	Clause types for the template builder's palette

TemplateCatalogueIntegration (`/catalogue/template/...`)

Endpoints	Purpose
`GET /catalogue/template/retrieve`, `GET /catalogue/template/retrieve/{did}`, `GET /catalogue/template/search`	List, retrieve by DID and search templates published in the Federated Catalogue (human contract roles)

ContractWorkflowEngine (`/contract/...`, `/machine-identities/...`)

Endpoint group	Endpoints	Purpose
Creation and maintenance	`POST /contract/create`, `PUT /contract/update`, `POST /contract/renew`	Create a contract from a template, edit it in draft, derive a follow-up contract
Offer	`POST /contract/offer`, `POST /contract/withdraw`, `POST /contract/submit`	Offer to the counterparty, withdraw before approval, submit a state transition
Negotiation	`POST /contract/negotiate`, `PUT /contract/negotiation_draft`, `GET/DELETE /contract/negotiation_draft/{did}`, `POST /contract/respond`	File change requests, park a negotiation draft, accept/reject a counter-proposal
Decision	`POST /contract/approve`, `POST /contract/reject`, `GET /contract/review`	Approve or reject, review view of open tasks
Retrieval and search	`GET /contract/retrieve`, `GET /contract/retrieve/{did}`, `GET /contract/{did}`, `GET /contract/history/{did}`, `GET /contract/search`, `GET /contract/templates`	List contracts, resolve by DID (party-restricted), history, search, available templates
Conclusion and operation	`POST /contract/store`, `POST /contract/terminate`, `GET /contract/kpis/{did}`, `POST /contract/audit`	Store evidence, terminate, view KPI observations, audit extract
Deployment	`POST /contract/deploy`, `POST /contract/target/designate`, `POST /contract/deployment/callback`	Deploy to the designated (or an explicitly chosen) target system; set or clear the designation; target system feedback (ack/status, KPI observations with verdict and rule reference, ADR-33), authenticated as its own OAuth2 client and only for deployments to exactly this target
Target system registry	`GET/POST/PUT/DELETE /contract/targets`, `POST /contract/targets/{id}/credential`	Administer the registry (write access Sys. Administrator and Integration Manager, read additionally Contract Manager); deletion is refused while a contract designates the target; issue/rotate the callback credential (secret visible exactly once)
Machine identities	`GET/POST /machine-identities`, `PUT/DELETE /machine-identities/{id}`, `POST /machine-identities/{id}/credential`	Manage the registry: creation provisions the OAuth2 client and returns the secret exactly once; deactivation rejects calls immediately; deletion also removes the client; rotation invalidates the old secret immediately (Sys. Administrator)

SignatureManagement (`/signature/...`)

Endpoint group	Endpoints	Purpose
Wallet ceremony	`POST /signature/request`, `GET /signature/request/{ceremony_id}`, `POST /signature/request/{ceremony_id}/publish`, `GET/POST /signature/request/{ceremony_id}/object`, `GET /signature/request/{ceremony_id}/document`, `GET /signature/request/{ceremony_id}/payload`, `POST /signature/request/{ceremony_id}/callback`	Start a ceremony, poll its status, deliver the request object, the PDF to be signed and the canonical JSON-LD payload to the wallet, receive the result via the nonce-bound callback
Direct signing flow	`POST /signature/prepare`, `POST /signature/submit`	Deliver the prepared, byte-pinned PDF and submit the finished external signature
Verification	`POST /signature/verify`, `POST /signature/validate`, `POST /signature/compliance`	Verify integrity and provenance, DSS-backed validation, level check
Retrieval and evidence	`GET /signature/retrieve`, `GET /signature/retrieve/{did}`, `GET /signature/provenance/{did}`, `GET /signature/view`, `GET /signature/audit`	Signing list, signature envelope per contract, provenance chain, compliance viewer, audit entries
Management	`POST /signature/revoke`	Revoke a signature

PDFGeneration (`/pdf/...`, export bundles)

Endpoint group	Endpoints	Purpose
PDF export	`GET /pdf/export/contract/{did}`, `GET /pdf/export/template/{did}`	Deterministically rendered PDF of a contract or template
Bundle export	`GET /contract/export/{did}`, `GET /template/export/{did}`	ZIP bundle with the PDF and machine-readable artifacts
Verification	`GET /pdf/verify/contract/{did}`, `GET /pdf/verify/template/{did}`	Independent re-render comparison with a named error class

ContractStorageArchive (`/archive/...`)

Endpoints	Purpose
`POST /archive/store`	Archive a concluded contract with its evidence
`GET /archive/retrieve` , `GET /archive/search`	Retrieve archive entries and search them structurally (full text, name, state, party, validity period, annotation tag)
`POST /archive/annotate`	Annotate an entry with a summary/tags
`DELETE /archive/delete`	Delete an entry with a justification: soft delete, removal of the snapshots and destruction of the content keys on both instances (ADR-28)
`GET /archive/erasure-status`	State of key destruction: local destruction records and open peer requests
`GET /archive/statistics`	Archive dashboard: inventory and storage statistics, recent actions, contracts expiring soon, evidence completeness
`GET /archive/audit`	Audit entries of the archive

ProcessAuditAndCompliance (`/pac/...`)

Endpoints	Purpose
`POST /pac/audit`	Audit retrieval: collect evidence and have it externally assessed (justification mandatory, retrieval is audited)
`GET /pac/report`	Generate an audit report (JSON/CSV/PDF)
`POST /pac/report`	Record an incident report for a resource
`GET /pac/monitor`	Compliance sweep on demand; a run without findings is audited as well
`GET /pac/audit/checkpoint/head` , `GET /pac/audit/checkpoint/{seq}` , `GET /pac/audit/checkpoint/proof/{entry_cid}`	Current Merkle checkpoint head, a specific checkpoint, inclusion proof for an audit entry
`GET /pac/workflow-gates/{run_id}` , `POST /pac/workflow-gates/review`	Read a workflow gate run; record a `REVIEW` decision with a justification

DcsToDcs (`/peer/...`)

Federation interface between two DCS instances. All endpoints stand behind the trust gate (agreement credential plus local policy endpoint, fail-closed) and are authenticated via did:web challenge-response in the request body, not via a session token.

Endpoints	Purpose
`POST /peer/contracts/pdf`	Receipt of a contract PDF with state, optionally JAdES, PoA evidence and the wrapped content key
`POST /peer/contracts/erase`	Erasure request: destruction of this contract's content keys on the receiving instance
`GET /peer/contracts/provenance`	Stored JAdES provenance of a received contract (JWT-protected, for local users)

KeyInventory (`/admin/hsm-keys`)

Endpoint	Purpose
`GET /admin/hsm-keys`	Read-only inventory: label, purpose and active version of each HSM key (Sys. Administrator). Rotation is an operational procedure, not an API action

C2PAService (`/c2pa/...`)

Endpoint	Purpose
`GET /c2pa/manifest/{contract_did}`	Public, unauthenticated retrieval of a contract's C2PA manifest store; optionally the chain listing

A.2 Events on the event bus

All backend domains publish their events as CloudEvents over a shared NATS topic, delivered via the transactional outbox pattern (Chapter 09). Consumers distinguish events by CloudEvent type.

Subscribers within the instance:

Consumer	Reacts to	Effect
DCS-to-DCS synchronizer	`OFFER_CONTRACT`, `PDF_REGENERATED`, `APPLIED_SIGNATURE`, `REVOKE_SIGNATURE`	Dispatch of the contract PDF to the peer instance for dispatch-relevant states
PDF/C2PA regenerator	Lifecycle events of contracts and templates	Background re-rendering and appending of a C2PA lifecycle assertion
Webhook platform	Domain events with a webhook mapping	Forwarding to registered external subscribers
Auto-deployment	`APPLIED_SIGNATURE`	Trigger of the target system delivery via the same command as the manual deploy
Event logger (optional)	all events	Diagnostic capture, only when explicitly enabled

The CloudEvent types follow the command vocabulary of their domain (`CREATE_CONTRACT`, `PUBLISH_CONTRACT_TEMPLATE`, `APPLIED_SIGNATURE`, `KEY_SHREDDED`, `PAC_COMPLIANCE_RISK`, …) and are at the same time the event type in the audit trail. The type on the bus is authoritative; pure read accesses produce lookup events that are not anchored in the trail (Chapter 09).

Webhook platform (`/orce/...`)

The webhook platform translates internal events into named, subscribable alert events: `contract.created`, `contract.submitted`, `contract.approved`, `contract.rejected`, `contract.negotiated`, `contract.terminated`, `contract.expired`, `template.created`, `template.approved`, `template.updated`, `template.registered`, `template.deprecated`, `compliance.risk_detected`.

Endpoint	Purpose
`GET /orce/events`	Catalog of subscribable event names
`GET /orce/webhooks`, `POST /orce/webhooks`, `DELETE /orce/webhooks/{id}`	List, register (event, callback URL, secret) and delete subscriptions
`POST /orce/callbacks`	Subscriber acknowledgement for a delivery
`GET /orce/deliveries`	Delivery log (result, status code, acknowledgement status)

Subscriptions and the delivery log live in process memory and do not survive a restart (Chapter 10).

A.3 Well-known endpoints

Public, unauthenticated endpoints at the origin root of the instance; the well-known documents are additionally reachable under the API prefix. They are the resolution basis of the federation trust model.

Endpoint	Content	Purpose
`GET /.well-known/did.json`	DID document of the instance (did:web)	Instance identity; public keys for the private keys held in the HSM, including the key agreement key
`GET /.well-known/dcs-agreement-credential.json`	Self-signed agreement credential	Proof of rule acceptance; subject of the trust gate check on both the dispatch and the receipt path
`GET /.well-known/dcs-federation-rules.md`	Federation rules	The rules document the agreement credential references by hash
`GET /status-list/{id}`	Signed status list (`application/statuslist+jwt`, x5c chain in the header)	Revocation status of the lifecycle and signing summary credentials issued by the instance; only at the origin root, only the registered list, responses not cacheable (Chapter 07)

In addition, for resolution by third parties:

- **OID4VP metadata** is not served as its own well-known document: the verifier metadata is contained in the signed authorization request object, anchored via the certificate chain in the JAR header and the DNS-bound client identifier.
- **OIDC discovery** is served by the co-operated OAuth2 provider, not by the DCS backend.
- `GET /c2pa/manifest/{contract_did}` is the public counterpart for resolving a contract's provenance.

- The **metrics endpoint** lies outside the authenticated API and outside the request budget (Chapter 10).

Appendix B Decision Archive

Reference list of the Architecture Decision Records under `docs/`. Each entry states the ADR's thesis and, where relevant, its relationship to other ADRs. Rationale and consequences are exclusively in the respective ADR. ADRs without a deviating note are in force unchanged.

Project series

ADR	Thesis and status
ADR-1	PKCS#11/HSM is the only key custody mechanism for all DCS-owned signing keys. Partially superseded by ADR-12: PAdES contract signatures are not DCS key custody touchpoints.
ADR-2	A single contract state machine with one transition table. The originally implied full-state-broadcast synchronizer is withdrawn by ADR-13.
ADR-3	Signing semantics: identity binding under the signature, embed-then-sign. The org-key-based AES over the PDF is withdrawn by ADR-12 and ADR-20.
ADR-4	C2PA provenance is delivered twice: embedded as JUMBF in the PDF and as a standards-conformant remote manifest.
ADR-5	Determination per XFSC component whether it is adopted or substituted; includes the graph store and the deployment lifecycle of the adopted Federated Catalogue.
ADR-6	Contract terms are real ODRL under a DCS profile and are enforced server-side. Only the evaluator mechanism is superseded by ADR-11.
ADR-7	Contract hierarchy links point exclusively from child to parent, never the other way.
ADR-8	Enforcement obtains SHACL shapes and validation profile exclusively from version-pinned Semantic Hub states.
ADR-9	Choice of the SHACL engine for semantic validation.
ADR-10	The clause catalog is transported as pre-digested JSON together with the raw Turtle in one response.
ADR-11	ODRL constraints are evaluated on embedded OPA. Supersedes the evaluator mechanism of ADR-6; the execution/KPI moment is in turn superseded by ADR-33, the pre-signing moment stands unchanged.
ADR-12	Contract signing is wallet-driven: the DCS is an OpenID4VP relying party and signature validator and holds no contract signing key. Supersedes the organization signature clause of ADR-3 and narrows ADR-1; its acceptance-path clauses are in turn superseded by ADR-20.
ADR-13	DCS-to-DCS federation is an exchange of contract PDF and JAdES, not a state broadcast; each instance runs workflow and RBAC locally. Supersedes the synchronizer of ADR-2; the third trust layer is superseded by ADR-19.
ADR-14	Internally, exactly one JSON-LD form should exist, expanded, with expansion only at the ingestion edges. Accepted as a decision, not implemented: the delivered internal form is the canonical `dcs:`-prefixed envelope (Chapter 03).
ADR-15	A negotiable data point is exactly one typed, SHACL-derived node linked by `@id` instead of a multi-level placeholder indirection. Superseded by ADR-22, which keeps the core and renames the node.
ADR-16	Tamper evidence of the audit trail arises via Merkle checkpoints per anchoring tick with external anchoring. Refines the event-sourced audit trail, whose per-resource hash chains remain.
ADR-17	A machine identity cannot produce an AES: the System Contract Signer class receives no signing scope at all. Extended with the seal option by ADR-21.
ADR-18	Towards the Federated Catalogue, the DCS deployment acts as one participant with one service account, not the individual publisher. A remote catalogue that is not part of the same deployment requires an explicit trust boundary confirmation, otherwise the catalogue integration fails.

ADR	Thesis and status
ADR-19	Federation trust is layered: did:web identity with certificate chain, rule acceptance via an agreement credential and a local policy endpoint (fail-closed). Replaces the third trust layer of ADR-13.
ADR-20	Hardening of the signature acceptance path: nonce binding of the ceremony callback, byte pinning of the document to be signed, atomic ceremony consumption, level-aware gates, certificate-to-identity check, JAdES validation. Supersedes the acceptance-path clauses of ADR-12.
ADR-21	System Contract Sealer: the electronic seal deferred by ADR-17, via the generic verify-on-reupload path instead of relaxing the AES prohibition. Accepted as a direction, not implemented.
ADR-22	The typed node is a field, not a placeholder: `dcs:contractFields` carries the field declarations, `dcs:contractData` typed domain objects with field references. Supersedes ADR-15; amended by ADR-23.
ADR-23	The contract data graph is generic: arbitrary SHACL vocabularies from the Semantic Hub, properties as literal, field reference or object reference, in-document resolution and validation of a field-materialized copy. Amends ADR-22, whose two-level form remains valid as a special case.
ADR-24	v1 signature scope: wallet-based AES; the achieved level is determined by the connected trust service, not the codebase. QES and further formats are documented as customer-confirmed deviations; the technical conformance gates remain.
ADR-25	Target systems are a persisted, administered registry, and every contract designates its own deployment target; a contract without a designation does not deploy. The callback authentication is superseded by ADR-27.
ADR-26	The signature is applied over the provenance, and the provenance is afterwards re-anchored over the signature (provenance-only C2PA update manifest as an incremental update); signature validity takes precedence in every conflict.
ADR-27	Machine credentials are issued, not configured: every machine caller receives its own client provisioned via the OAuth2 provider with a secret shown exactly once; the deployment declaration is reduced to a seed. Supersedes the callback authentication of ADR-25.
ADR-28	Every stored artifact is encrypted under a random key per erasure scope, the key exists at rest only HSM-wrapped, and Art. 17 erasure is the recorded destruction of these copies on both federation instances.
ADR-29	The organizational authority of a power of attorney is bound to its issuer; the Integration Manager role covers integrations, not access.
ADR-30	An artifact that fails authenticated decryption is a negative verification verdict, not a server error; the verification result names its error class directly.
ADR-31	Issuer trust is purpose-scoped (`login` / `peer` / `pid`), bound to organizations and resolved via a declared mechanism; the power of attorney behind a signature travels along and is verified by the counterparty.
ADR-32	Continuous compliance monitoring runs as a scheduled sweep via a dedicated command instead of a loop in the server, with a risk register to deduplicate the alerts.
ADR-33	The target system classifies contract fulfilment at runtime; the DCS records the reported verdicts, attributes them to the ODRL rule and audits them instead of deriving them itself. Supersedes the execution/KPI moment of ADR-11.
ADR-34	A status list is signed and served by the issuer whose credential it governs, and verified exactly like the credential itself; an unsigned status list is accepted under no configuration. By amendment, the DCS serves the list of its self-issued credentials itself.

ADR	Thesis and status
ADR (OCM-W)	VC signing runs directly via the HSM, not via the OCM-W services. The only unnumbered ADR of the series.

Backend-internal series

Older, backend-local decisions under `docs/backend/en/decisions/`. They describe base patterns the project series builds on.

ADR	Thesis
0001	CQRS split per business domain (command/query/db/datatype/event).
0002	Design-first API with generated transport.
0003	Event-sourced audit trail with per-resource hash chains, refined by ADR-16.
0004	did:web identity with an eIDAS certificate chain as trust anchor.
0005	One writer per contract (the origin instance); the transport form is superseded by ADR-13.
0006	Versioning strategies for templates and contracts.
0007	Optimistic concurrency control via the change timestamp.